

Thiết kế bộ tăng tốc phần cứng cho FFT Radix-2/Radix-4

Sinh viên: *[Điền tên sinh viên]*

Giảng viên hướng dẫn: *[Điền tên giảng viên]*

Ngày 1 tháng 6 năm 2026

Mục lục

1	Giới thiệu đề tài	4
2	Đặt vấn đề	5
2.1	Bài toán DFT	6
2.2	Thuật toán FFT	6
2.3	Thuật toán Radix-2 và Radix-4	7
2.3.1	Radix-2 FFT	7
2.3.2	Radix-4 FFT	8
2.3.3	So sánh Radix-2 và Radix-4	9
3	Một số kiến trúc FFT	10
3.1	Mục tiêu của kiến trúc FFT pipeline	10
3.2	R2MDC – Radix-2 Multi-path Delay Commutator	11
3.3	R2SDF – Radix-2 Single-path Delay Feedback	11
3.4	R4SDF – Radix-4 Single-path Delay Feedback	12
3.5	R4MDC – Radix-4 Multi-path Delay Commutator	13
3.6	R4SDC – Radix-4 Single-path Delay Commutator	14
3.7	So sánh tài nguyên phần cứng	14
3.8	Lựa chọn hướng thiết kế	15
4	Thuật toán Radix-2 bình phương DIF FFT	17
4.1	Mục tiêu và giả thiết của thuật toán Radix-2 bình phương	17
4.2	Định nghĩa DFT và ánh xạ chỉ số	17
4.3	Phân rã Radix-2 DIF thứ nhất	18
4.4	Phân rã hệ số xoay tổng hợp	19
4.5	Butterfly thứ hai và phương trình H	20
4.6	Ý nghĩa phần cứng của các hệ số xoay	20
4.7	Tóm tắt đặc điểm thuật toán và liên hệ với kiến trúc	22

5	Thiết kế RTL kiến trúc Radix-2 bình phương SDF FFT	23
5.1	Tham số hệ thống	23
5.2	Từ phương trình thuật toán sang khối RTL	24
5.3	Sơ đồ khối tổng thể FFT-256 R2 bình phương SDF	25
5.4	Nguyên lý tầng SDF	26
5.5	Thiết kế BF2I – butterfly Radix-2	27
5.6	Thiết kế BF2II và bộ xoay tầm thường	28
5.7	Thiết kế bộ nhân phức CMUL	29
5.8	Bộ nhớ hệ số xoay và sinh địa chỉ	30
5.9	Đường trễ và bộ nhớ phản hồi	31
5.10	Bộ điều khiển	32
5.11	Ước lượng tài nguyên	33
5.12	Tổng kết chương	34
6	Triển khai RTL và kiểm chứng chức năng FFT-256 R2 bình phương SDF	35
6.1	Mục tiêu và phạm vi chương	35
6.2	Quy ước giao tiếp RTL và định dạng dữ liệu	36
6.3	Các khối RTL cơ bản	37
6.3.1	Khối xoay pha tầm thường <code>trivial_rotator.sv</code>	37
6.3.2	Butterfly Radix-2 <code>butterfly_radix2_scaled.sv</code>	37
6.3.3	Bộ nhân phức Q1.15 <code>complex_multiplier_q15.sv</code>	38
6.3.4	Đường trễ tuyến tính và đường trễ phản hồi	39
6.3.5	Bộ nhớ hệ số xoay	40
6.4	Tầng SDF và khối Radix-2 ² SDF	40
6.4.1	Tầng BF2I	40
6.4.2	Tầng BF2II và <code>trivial_rotator</code>	41
6.4.3	Khối Radix-2 ² SDF có điều khiển cục bộ	42
6.4.4	Sinh địa chỉ hệ số xoay	42
6.5	Từ FFT-16 thử nghiệm đến FFT-256 hoàn chỉnh	43
6.5.1	Bộ FFT-16 dùng để kiểm tra nhanh	43
6.5.2	Bộ FFT-256	44
6.6	Kiểm chứng chức năng RTL	45
6.7	Quan sát dạng sóng bằng GTKWave	47
6.8	Giới hạn hiện tại và hướng hoàn thiện	47

6.9	Tổng kết chương	47
-----	---------------------------	----

Chương 1

Giới thiệu đề tài

Biến đổi Fourier rời rạc (Discrete Fourier Transform – DFT) là phép toán nền tảng để chuyển tín hiệu rời rạc từ miền thời gian sang miền tần số. Trong các hệ thống xử lý tín hiệu số như radar, viễn thông số, OFDM, xử lý ảnh, âm thanh và đo lường phổ, DFT thường không được tính trực tiếp vì chi phí tính toán tăng rất nhanh theo số điểm biến đổi. Thay vào đó, hệ thống sử dụng Fast Fourier Transform (FFT) họ thuật toán giúp tính DFT với số phép toán thấp hơn đáng kể.

Mục tiêu của đề tài là triển khai thuật toán FFT dưới dạng bộ tăng tốc phần cứng, thiết kế một kiến trúc xử lý dữ liệu theo pipeline, giảm số bộ nhân phức, giảm số bộ nhớ trễ mà vẫn giữ được độ chính xác chấp nhận được.

Cụ thể hóa, dự án đi vào các nội dung từ gốc rễ đến nâng cao: Giải quyết bài toán DFT và thuật toán FFT, hai nhóm thuật toán phổ biến Radix-2/Radix-4, một số kiến trúc Pipelined FFT được sử dụng rộng rãi, và nghiên cứu kiến trúc Radix-2² Single-path Delay Feedback (R2²SDF).

Chương 2

Đặt vấn đề

Với DFT trực tiếp, mỗi điểm phổ $X[k]$ là tổng có trọng số của toàn bộ N mẫu đầu vào. Nếu cần tính đủ N điểm phổ, hệ thống phải thực hiện xấp xỉ N^2 phép nhân phức và $N(N-1)$ phép cộng phức. Với N nhỏ, cách làm này vẫn có thể chấp nhận được. Nhưng khi N tăng lên 256, 1024, 4096 hoặc lớn hơn, chi phí phần cứng và thời gian xử lý trở nên rất nặng.

Các ứng dụng viễn thông thường cần biến đổi FFT liên tục theo từng frame dữ liệu. Ví dụ, trong xử lý radar FMCW, FFT được dùng để tách thành phần tần số beat, từ đó suy ra khoảng cách hoặc vận tốc. Trong hệ thống OFDM, FFT/IFFT là lõi xử lý để chuyển đổi giữa miền tần số và miền thời gian. Những bài toán này không chỉ cần kết quả đúng, mà còn cần thông lượng đủ cao để không làm nghẽn toàn bộ chuỗi xử lý.

Khi đưa FFT lên phần cứng, vấn đề chính không chỉ là giảm số phép toán trên giấy. Phần cứng phải trả giá cho từng bộ nhân phức, từng bộ cộng phức, từng thanh ghi trễ và từng khối điều khiển địa chỉ. Trong thiết kế VLSI/FPGA, bộ nhân phức thường là tài nguyên đắt nhất. Bộ nhớ trễ cũng có thể trở thành phần chiếm diện tích lớn khi số điểm FFT tăng. Vì vậy, kiến trúc FFT tốt phải tối ưu đồng thời số bộ nhân, số bộ cộng, bộ nhớ và độ phức tạp điều khiển.

Do đó, đề án tập trung giải quyết ba câu hỏi thiết kế chính:

1. Chọn thuật toán FFT nào để giảm số phép nhân nhưng vẫn giữ cấu trúc phần cứng đủ đơn giản?
2. Chọn kiến trúc pipeline nào để xử lý luồng dữ liệu liên tục với bộ nhớ trễ nhỏ?
3. Quản lý số cố định, độ rộng dữ liệu, tràn số và chia tỷ lệ như thế nào để kết quả đúng trong giới hạn tài nguyên?

Trong phạm vi chương này sẽ tập trung trả lời câu hỏi thứ nhất và câu hỏi thứ 2. Kiểm chứng RTL và thực nghiệm sẽ được thực hiện ở các chương sau.

2.1 Bài toán DFT

DFT N điểm của chuỗi đầu vào $x[n]$ được định nghĩa như sau:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{nk}, \quad 0 \leq k < N, \quad (2.1)$$

trong đó hệ số xoay cơ sở là

$$W_N = e^{-j\frac{2\pi}{N}}. \quad (2.2)$$

W_N biểu diễn phép quay pha trên mặt phẳng phức. Dấu $-j$ thể hiện chiều quay dùng cho DFT thuận theo quy ước phổ biến trong xử lý tín hiệu số. Nếu cần khôi phục tín hiệu thời gian từ phổ, IDFT được viết là

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-nk}. \quad (2.3)$$

Về mặt toán học, DFT là phép chiếu chuỗi $x[n]$ lên các hàm cơ sở phức $e^{-j2\pi kn/N}$. Mỗi chỉ số k tương ứng một bin tần số. Khi N tăng, độ phân giải tần số tăng, nhưng số phép toán của DFT trực tiếp tăng theo bậc hai.

2.2 Thuật toán FFT

FFT là họ thuật toán tính DFT nhanh bằng cách chia bài toán N điểm thành nhiều DFT con nhỏ hơn. Ý tưởng phổ biến nhất là thuật toán Cooley–Tukey: nếu N có thể phân tích thành tích của các thừa số, DFT N điểm có thể được tách thành nhiều DFT kích thước nhỏ hơn, sau đó ghép lại bằng các butterfly và hệ số xoay [2].

Về mặt kiến trúc, FFT có thể triển khai theo hai hướng phân rã chính: Decimation-In-Time (DIT) và Decimation-In-Frequency (DIF). DIT tách chuỗi đầu vào theo chỉ số thời gian trước, còn DIF tách kết quả tần số trước. Hai dạng này cho cùng kết quả DFT nhưng khác thứ tự dữ liệu, vị trí nhân hệ số xoay và cách ánh xạ sang phần cứng.

Điểm quan trọng là FFT không làm biến mất phép nhân; nó sắp xếp lại phép nhân để số lượng và vị trí phần cứng trở nên hợp lý hơn. Trong bộ tăng tốc phần cứng, điều này cực kỳ quan trọng vì một bộ nhân phức không phải chi phí nhỏ. Khi thiết kế RTL, cần nhìn FFT dưới dạng các khối:

1. số bộ nhân phức cần dùng;
2. số bộ cộng/trừ phức trong butterfly;

3. kích thước bộ nhớ trễ hoặc thanh ghi phản hồi;
4. độ phức tạp của điều khiển, bao gồm bộ đếm, mạng chuyển mạch, bộ nhớ hệ số xoay và thứ tự dữ liệu.

Một tầng butterfly Radix-2 ở dạng DIF có thể viết gọn như sau, với a và b là hai mẫu cách nhau một khoảng xác định bởi tầng xử lý:

$$y_0 = a + b, \quad (2.4)$$

$$y_1 = (a - b)W_N^k. \quad (2.5)$$

Hai phép cộng/trừ tạo thành lõi butterfly. Nhánh y_1 cần nhân hệ số xoay, trừ một số trường hợp hệ số là $1, -1, j$ hoặc $-j$. Các hệ số này gọi là phép nhân tầm thường vì có thể thực hiện bằng đổi dấu hoặc hoán vị phần thực/phần ảo thay vì dùng bộ nhân phức đầy đủ.

2.3 Thuật toán Radix-2 và Radix-4

2.3.1 Radix-2 FFT

Radix-2 là dạng FFT dễ hiểu và dễ triển khai nhất khi $N = 2^m$. Với DIT, chuỗi đầu vào được tách thành nhóm chỉ số chẵn và lẻ:

$$E[k] = \sum_{m=0}^{N/2-1} x[2m]W_{N/2}^{mk}, \quad (2.6)$$

$$O[k] = \sum_{m=0}^{N/2-1} x[2m+1]W_{N/2}^{mk}. \quad (2.7)$$

Kết quả DFT N điểm được ghép lại từ hai DFT $N/2$ điểm:

$$X[k] = E[k] + W_N^k O[k], \quad (2.8)$$

$$X\left[k + \frac{N}{2}\right] = E[k] - W_N^k O[k]. \quad (2.9)$$

Cặp công thức trên cho thấy một DFT N điểm được chia thành hai DFT $N/2$ điểm. Quá trình này lặp lại cho đến khi chỉ còn DFT 2 điểm, tức là phép cộng và trừ đơn

giản. Ưu điểm của Radix-2 là butterfly nhỏ, đều, dễ triển khai và dễ kiểm chứng. Nhược điểm là số tầng bằng $\log_2 N$, nên với N lớn, số tầng và số vị trí nhân hệ số xoay tăng.

Với phần cứng số cố định, Radix-2 cũng dễ quản lý chia tỷ lệ hơn vì mỗi tầng chỉ cộng/trừ hai số phức. Tuy nhiên, nếu mục tiêu là giảm bộ nhân phức, Radix-2 nguyên bản chưa phải lựa chọn tối ưu. Đây là lý do các kiến trúc Radix-4 và Radix-2² được quan tâm.

2.3.2 Radix-4 FFT

Radix-4 phân rã DFT N điểm thành bốn DFT $N/4$ điểm, phù hợp khi $N = 4^m$. Một butterfly Radix-4 xử lý bốn đầu vào a, b, c, d . Phần cộng/trừ được viết dưới dạng:

$$Y_0 = a + b + c + d, \quad (2.10)$$

$$Y_1 = a - jb - c + jd, \quad (2.11)$$

$$Y_2 = a - b + c - d, \quad (2.12)$$

$$Y_3 = a + jb - c - jd. \quad (2.13)$$

Sau phần cộng/trừ, các nhánh thường tiếp tục nhân với các hệ số xoay $W_N^k, W_N^{2k}, W_N^{3k}$ tùy tầng và vị trí. So với Radix-2, Radix-4 giảm số tầng từ $\log_2 N$ xuống $\log_4 N$, tức giảm một nửa số tầng theo lý thuyết. Vì vậy, Radix-4 thường có lợi về số phép nhân không tầm thường.

Tuy nhiên, lợi ích này không miễn phí. Butterfly Radix-4 phức tạp hơn butterfly Radix-2 vì nó xử lý bốn đầu vào cùng lúc và cần nhiều bộ cộng/trừ hơn. Trong RTL, butterfly lớn hơn có thể làm định thời khó hơn, điều khiển nặng hơn và việc gỡ lỗi trên dạng sóng cũng rối hơn.

Nói gọn: Radix-2 thân thiện với phần cứng vì butterfly đơn giản; Radix-4 có lợi về số bộ nhân và số tầng nhưng butterfly nặng hơn. Khi lựa chọn, không nên chọn thuật toán chỉ vì “ít tầng hơn”. Phải nhìn cả đường dữ liệu, bộ nhớ, điều khiển và độ rủi ro khi triển khai.

2.3.3 So sánh Radix-2 và Radix-4

Bảng 2.1: So sánh định hướng phần cứng giữa Radix-2 và Radix-4

Tiêu chí	Radix-2	Radix-4
Điều kiện N	$N = 2^m$	$N = 4^m$
Số tầng	$\log_2 N$	$\log_4 N$.
Butterfly	Đơn giản, 2 đầu vào/2 đầu ra.	Phức tạp hơn, 4 đầu vào/4 đầu ra.
Bộ nhân hệ số xoay	Nhiều vị trí nhân hơn.	Ít vị trí nhân không tầm thường hơn.
Ưu điểm phần cứng	Dễ pipeline, dễ kiểm chứng, điều khiển đều.	Giảm số tầng và có thể giảm số bộ nhân.
Nhược điểm phần cứng	Nhiều tầng hơn, số nhân có thể lớn.	Butterfly lớn, nhiều bộ cộng/trừ, điều khiển khó hơn.

Chương 3

Một số kiến trúc FFT

Sau khi chọn thuật toán, bước tiếp theo là ánh xạ thuật toán sang kiến trúc phần cứng. Với FFT pipeline, dữ liệu đi qua nhiều tầng xử lý liên tiếp; mỗi tầng chứa butterfly, bộ nhân hệ số xoay, thanh ghi trễ và khối điều khiển. Kiến trúc pipeline phù hợp với xử lý thời gian thực vì sau khi pipeline được lấp đầy, hệ thống có thể nhận mẫu mới và xuất kết quả liên tục.

Có hai cặp khái niệm phải nắm rõ trước khi đọc các kiến trúc:

- **Đa đường dữ liệu và một đường dữ liệu (Multi-path and Single-path):** kiến trúc đa đường chia luồng dữ liệu thành nhiều nhánh song song; kiến trúc một đường giữ một luồng dữ liệu chính đi qua các tầng. Kiến trúc đa đường có thể tăng mức song song nhưng thường cần mạng chuyển mạch và bộ nhớ sắp xếp dữ liệu phức tạp hơn.
- **Chuyển mạch trễ và phản hồi trễ (Delay Commutator and Delay Feedback):** kiến trúc chuyển mạch trễ dùng bộ trễ để sắp xếp lại thời điểm dữ liệu gặp nhau ở butterfly; kiến trúc phản hồi trễ đưa một phần kết quả butterfly quay ngược về thanh ghi phản hồi để sử dụng ở chu kỳ sau. Theo một số nghiên cứu, nhóm phản hồi trễ thường hiệu quả bộ nhớ hơn nhóm chuyển mạch trễ tương ứng vì dữ liệu lưu trong phản hồi có thể được dùng trực tiếp bởi các tầng sau [1].

3.1 Mục tiêu của kiến trúc FFT pipeline

Một FFT pipeline tốt cần cân bằng thông lượng, độ trễ, tài nguyên và độ phức tạp điều khiển. Trong bài toán thời gian thực, thông lượng thường quan trọng hơn độ trễ tuyệt đối: hệ thống có thể chịu được việc kết quả ra muộn vài trăm hoặc vài nghìn chu kỳ, miễn là sau khi pipeline ổn định, mỗi chu kỳ vẫn xử lý được dữ liệu mới.

Các thành phần tiêu tốn tài nguyên lớn nhất gồm:

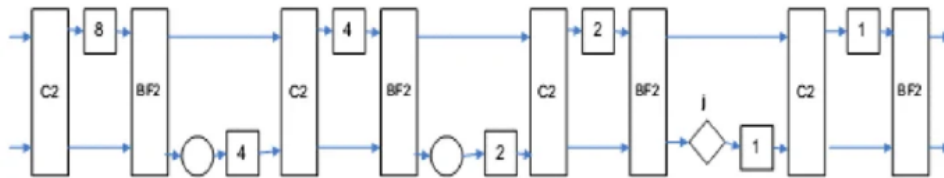
- **Bộ nhân phức:** thường đắt nhất về diện tích phần cứng và định thời. Một bộ nhân phức trực tiếp cần nhiều phép nhân thực và cộng/trừ thực.
- **Bộ nhớ trễ/thanh ghi dịch:** cần để giữ dữ liệu cho đúng yêu cầu của bộ butterfly. Với N lớn, đây có thể là phần chiếm diện tích đáng kể.
- **Butterfly:** gồm cộng/trừ phức, đổi dấu, hoán vị phần thực/phần ảo và đôi khi tích hợp nhân tầm thường.
- **Khối điều khiển:** gồm bộ đếm, chọn MUX, cấp phát địa chỉ bộ nhớ hệ số xoay, cho phép pipeline và xử lý thứ tự dữ liệu.

Vì vậy, không thể đánh giá kiến trúc chỉ bằng số bộ nhân. Một kiến trúc ít nhân nhưng cần butterfly quá lớn hoặc điều khiển quá phức tạp vẫn có thể không phù hợp.

3.2 R2MDC – Radix-2 Multi-path Delay Commutator

R2MDC là cách ánh xạ tương đối trực tiếp từ thuật toán Radix-2 sang pipeline. Dữ liệu đầu vào được chia thành hai luồng song song. Các đường trễ và bộ chuyển mạch được dùng để bảo đảm hai mẫu cần ghép butterfly xuất hiện đúng cùng thời điểm. Theo các nghiên cứu, R2MDC cần $2(\log_4 N - 1)$ bộ nhân phức, $4 \log_4 N$ bộ cộng phức và $3N/2 - 2$ thanh ghi trễ phức [1].

Ưu điểm của R2MDC là dễ hiểu và điều khiển tương đối đơn giản. Nhược điểm là chia nhiều luồng khiến bộ trễ và bộ chuyển mạch tăng lên. Với mục tiêu tối ưu tài nguyên, R2MDC không phải hướng mạnh nhất, nhưng nó rất hữu ích để làm kiến trúc nền khi bắt đầu tiếp cận FFT pipeline.



Hình 3.1: Sơ đồ khối R2MDC.

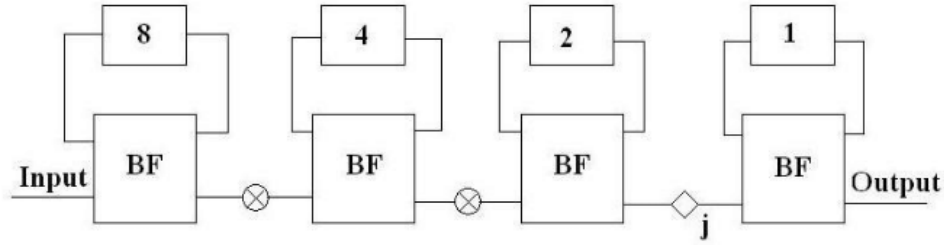
3.3 R2SDF – Radix-2 Single-path Delay Feedback

R2SDF cải thiện R2MDC bằng cách giữ một luồng dữ liệu duy nhất và dùng thanh ghi phản hồi để lưu kết quả butterfly. Một phần kết quả được đưa tiếp, phần còn lại quay

về đường trễ để dùng ở chu kỳ thích hợp. Cách làm này tận dụng bộ nhớ tốt hơn vì thanh ghi phản hồi không chỉ là phần tử chờ, mà là một phần trực tiếp của luồng tính toán.

R2SDF có cùng số butterfly và bộ nhân như R2MDC, nhưng bộ nhớ giảm còn $N - 1$ thanh ghi phức, được xem là mức tối thiểu đối với nhóm kiến trúc FFT pipeline dựa trên thuật toán này [1]. Đây là lý do SDF thường được xem là hướng hấp dẫn khi mục tiêu là giảm bộ nhớ.

Điểm yếu của R2SDF là vẫn mang đặc điểm Radix-2: số tầng nhiều và số bộ nhân chưa thấp bằng các hướng dựa trên Radix-4.

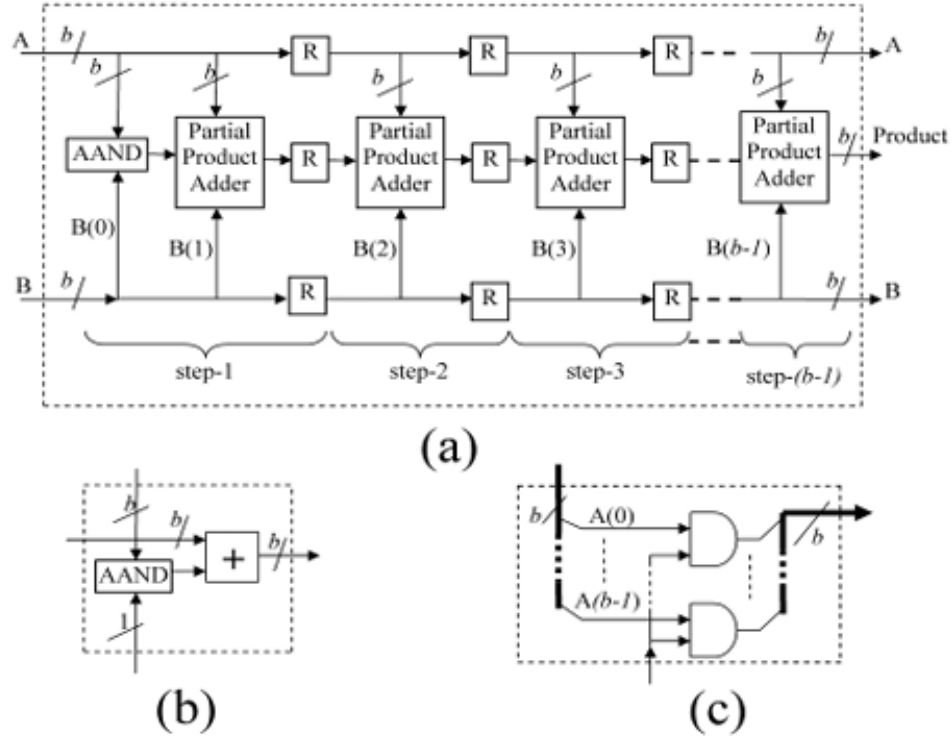


Hình 3.2: Sơ đồ khối R2SDF.

3.4 R4SDF – Radix-4 Single-path Delay Feedback

R4SDF là phiên bản Radix-4 của SDF. Vì Radix-4 xử lý bốn mẫu một nhóm, số tầng giảm còn $\log_4 N$. Theo nghiên cứu, mức sử dụng bộ nhân của R4SDF tăng lên khoảng 75% nhờ cơ chế lưu ba trong bốn đầu ra butterfly, nhưng mức sử dụng butterfly Radix-4 chỉ khoảng 25% vì butterfly đầy đủ khá lớn và không luôn hoạt động hết công suất [1].

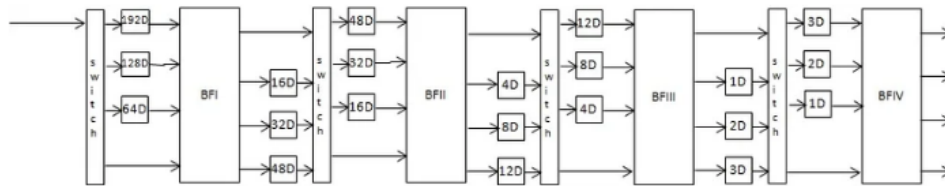
R4SDF có điểm mạnh rõ ràng: bộ nhớ vẫn tốt như R2SDF và số bộ nhân giảm. Nhưng cái giá là butterfly nặng hơn. Một butterfly Radix-4 đầy đủ có thể cần ít nhất 8 bộ cộng, tức là phần cộng/trừ phức không còn nhẹ như Radix-2. Với thiết kế RTL, điều này có thể làm tăng đường tới hạn và khiến kiểm chứng từng tầng khó hơn.



Hình 3.3: Sơ đồ khối R4SDF.

3.5 R4MDC – Radix-4 Multi-path Delay Commutator

R4MDC là phiên bản Radix-4 theo hướng đa đường dữ liệu dùng chuyển mạch trễ. Nó từng được dùng trong các triển khai FFT pipeline VLSI ban đầu, nhưng có nhược điểm lớn là mức sử dụng tài nguyên thấp. Theo bài nghiên cứu, các thành phần trong R4MDC chỉ đạt khoảng 25% mức sử dụng trong trường hợp thông thường, trừ các ứng dụng đặc biệt xử lý 4 FFT song song [1].

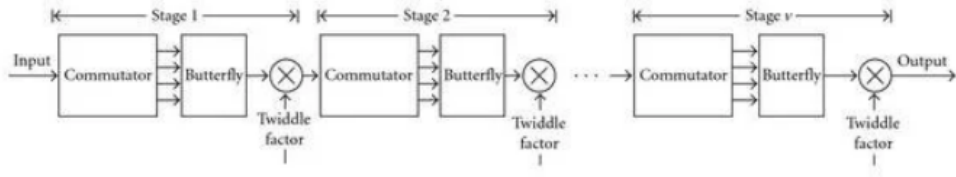


Hình 3.4: Sơ đồ khối R4MDC.

3.6 R4SDC – Radix-4 Single-path Delay Commutator

R4SDC sử dụng thuật toán Radix-4 đã chỉnh sửa và các butterfly 1/4 Radix-4 có khả năng lập trình để tăng tối ưu hệ số nhân lên khoảng 75%. Kiến trúc này dùng chuyển mạch trễ kết hợp để giảm bộ nhớ so với R4MDC, từ mức $5N/2 - 4$ xuống còn $2N - 2$. [1].

Tuy nhiên, khả năng điều khiển của R4SDC được đánh giá là rất phức tạp. Vì vậy, R4SDC tốt trên lý thuyết tài nguyên nhưng không phải lựa chọn dễ triển khai.



Hình 3.5: Sơ đồ khối R4SDC.

3.7 So sánh tài nguyên phần cứng

Bảng 3.1 tổng hợp so sánh tài nguyên. Các công thức được quy về $\log_4 N$ để so sánh Radix-2, Radix-4 và Radix- 2^2 trong cùng một hệ quy chiếu. Giả thiết chính của bảng là N là lũy thừa của 4 và dữ liệu/các phép toán đều là số phức.

Bảng 3.1: So sánh tài nguyên một số kiến trúc FFT pipeline [1]

Kiến trúc	Số bộ nhân phức	Số bộ cộng phức	Số bộ nhớ trễ	Điều khiển	Nhận xét
R2MDC	$2(\log_4 N - 1)$	$4 \log_4 N$	$3N/2 - 2$	Đơn giản	Trực tiếp từ Radix-2; bộ nhớ lớn hơn SDF; mức sử dụng tài nguyên thấp.
R2SDF	$2(\log_4 N - 1)$	$4 \log_4 N$	$N - 1$	Đơn giản	Bộ nhớ tối thiểu; butterfly đơn giản; số bộ nhân còn cao.
R4SDF	$\log_4 N - 1$	$8 \log_4 N$	$N - 1$	Trung bình	Ít nhân và bộ nhớ tốt; butterfly Radix-4 nặng.
R4MDC	$3(\log_4 N - 1)$	$8 \log_4 N$	$5N/2 - 4$	Đơn giản	Nhiều đường dữ liệu; mức sử dụng tài nguyên thấp; bộ nhớ lớn.
R4SDC	$\log_4 N - 1$	$3 \log_4 N$	$2N - 2$	Phức tạp	Ít nhân, ít cộng; bộ chuyển mạch/điều khiển khó.
R2 ² SDF	$\log_4 N - 1$	$4 \log_4 N$	$N - 1$	Đơn giản	Kết hợp ưu điểm: số nhân như Radix-4, butterfly kiểu Radix-2, bộ nhớ SDF.

3.8 Lựa chọn hướng thiết kế

Từ bảng so sánh, có thể rút ra vài kết luận thiết kế quan trọng. Thứ nhất, nhóm phản hồi trễ tốt hơn chuyển mạch trễ về bộ nhớ trong các kiến trúc cùng radix. R2SDF giảm bộ nhớ từ $3N/2 - 2$ xuống $N - 1$ so với R2MDC. R4SDF cũng giữ mức $N - 1$ trong khi R4MDC cần $5N/2 - 4$.

Thứ hai, Radix-4 giúp giảm số bộ nhân nhưng không miễn phí. R4SDF chỉ cần

$\log_4 N - 1$ bộ nhân, nhưng số bộ cộng phức trong butterfly tăng mạnh. R4SDC nhìn hấp dẫn vì ít bộ cộng hơn, nhưng phần điều khiển phức tạp.

Thứ ba, Radix- 2^2 là hướng hợp lý để nối giữa Radix-2 và Radix-4. Theo nghiên cứu, Radix- 2^2 có cùng độ phức tạp nhân với Radix-4 nhưng giữ butterfly kiểu Radix-2 [1]. Khi ánh xạ vào SDF, R 2^2 SDF đạt đồng thời ba điểm mạnh: số bộ nhân như Radix-4, bộ nhớ $N - 1$ như SDF và butterfly đơn giản kiểu Radix-2. Đây là lý do nó là ứng viên rất phù hợp cho bộ tăng tốc phần cứng FFT.

Chương 4

Thuật toán Radix-2² DIF FFT

4.1 Mục tiêu và giả thiết của thuật toán Radix-2²

Thuật toán Radix-2² được He và Torkelson xây dựng cho FFT pipeline thời gian thực theo dạng Decimation-In-Frequency (DIF). Mục tiêu của thuật toán là giữ cấu trúc butterfly đơn giản của Radix-2, nhưng sắp xếp lại vị trí các phép nhân hệ số xoay để số phép nhân phức không tầm thường tương đương Radix-4 [1].

Điểm chính của Radix-2² là xét đồng thời hai bước phân rã Radix-2 DIF liên tiếp. Sau hai bước này, hệ số xoay tổng hợp được tách thành một phần tầm thường và một phần không tầm thường. Phần tầm thường chỉ nhận các giá trị $1, -1, j, -j$, nên có thể hiện thực bằng đổi dấu và hoán đổi phần thực-ảo thay vì dùng bộ nhân phức đầy đủ. Đây là lý do thuật toán này phù hợp với thiết kế bộ tăng tốc phần cứng: giảm số bộ nhân phức nhưng vẫn giữ đường dữ liệu đều, dễ xử lý theo pipeline.

Trong chương này, giả thiết N là lũy thừa của 4, tức $N = 4^m$. Dữ liệu đầu vào được xét theo thứ tự tự nhiên. Kết quả đầu ra của dạng DIF pipeline thường xuất hiện ở thứ tự đảo chữ số theo radix tương ứng; việc sắp xếp lại đầu ra, nếu cần, sẽ được xử lý ở các tầng sau.

4.2 Định nghĩa DFT và ánh xạ chỉ số

Như đã nhắc ở trên, DFT được định nghĩa là

$$X[k] = \sum_{n=0}^{N-1} x[n]W_N^{nk}, \quad 0 \leq k < N, \quad (4.1)$$

trong đó

$$W_N = e^{-j\frac{2\pi}{N}} \quad (4.2)$$

là căn nguyên thủy bậc N của đơn vị. Các số mũ của W_N được hiểu theo modulo N , nghĩa là $W_N^{a+N} = W_N^a$.

Để xét hai bước Radix-2 DIF đầu tiên trong cùng một cụm, chỉ số thời gian n và chỉ số tần số k được ánh xạ thành

$$n = \frac{N}{2}n_1 + \frac{N}{4}n_2 + n_3, \quad (4.3)$$

$$k = k_1 + 2k_2 + 4k_3, \quad (4.4)$$

trong đó

$$n_1, n_2, k_1, k_2 \in \{0, 1\}, \quad 0 \leq n_3, k_3 < \frac{N}{4}. \quad (4.5)$$

Ý nghĩa của các biến như sau. Biến n_1 chọn nửa đầu hoặc nửa sau của chuỗi vào, nên nó tạo cặp mẫu cách nhau $N/2$ ở butterfly Radix-2 thứ nhất. Biến n_2 chọn một trong hai phần tư bên trong mỗi nửa, nên nó tạo cặp mẫu cách nhau $N/4$ ở butterfly Radix-2 thứ hai. Biến n_3 là chỉ số nội bộ trong mỗi DFT con chiều dài $N/4$. Ở phía tần số, k_1 và k_2 là hai bit thấp của chỉ số k , còn k_3 là phần chỉ số còn lại của DFT con.

4.3 Phân rã Radix-2 DIF thứ nhất

Thay (4.3) và (4.4) vào định nghĩa DFT (4.1), ta có

$$\begin{aligned} X[k_1 + 2k_2 + 4k_3] &= \sum_{n_3=0}^{N/4-1} \sum_{n_2=0}^1 \sum_{n_1=0}^1 x \left[\frac{N}{2}n_1 + \frac{N}{4}n_2 + n_3 \right] \\ &\quad \times W_N^{\left(\frac{N}{2}n_1 + \frac{N}{4}n_2 + n_3\right)(k_1 + 2k_2 + 4k_3)}. \end{aligned} \quad (4.6)$$

Phần hệ số xoay phụ thuộc vào n_1 là

$$W_N^{\frac{N}{2}n_1(k_1 + 2k_2 + 4k_3)} = e^{-j\pi n_1(k_1 + 2k_2 + 4k_3)}. \quad (4.7)$$

Vì $2k_2 + 4k_3$ luôn là số chẵn, chỉ có k_1 quyết định dấu của biểu thức trên. Do đó

$$W_N^{\frac{N}{2}n_1(k_1 + 2k_2 + 4k_3)} = (-1)^{n_1 k_1}. \quad (4.8)$$

Đặt

$$r = \frac{N}{4}n_2 + n_3. \quad (4.9)$$

Tổng theo n_1 tạo ra butterfly Radix-2 DIF thứ nhất:

$$\mathcal{B}_N^{k_1}(r) = x[r] + (-1)^{k_1} x \left[r + \frac{N}{2} \right]. \quad (4.10)$$

Với $k_1 = 0$, đây là nhánh cộng; với $k_1 = 1$, đây là nhánh trừ.

Sau butterfly thứ nhất, DFT trở thành

$$X[k_1 + 2k_2 + 4k_3] = \sum_{n_3=0}^{N/4-1} \sum_{n_2=0}^1 \mathcal{B}_N^{k_1} \left(\frac{N}{4} n_2 + n_3 \right) \times W_N^{\left(\frac{N}{4} n_2 + n_3 \right) (k_1 + 2k_2 + 4k_3)}. \quad (4.11)$$

Nếu tiếp tục phân rã ngay theo cách thông thường, ta thu được Radix-2 DIF. Radix-2² khác ở chỗ tiếp tục phân tích hệ số xoay còn lại trước khi xây butterfly thứ hai.

4.4 Phân rã hệ số xoay tổng hợp

Xét hệ số xoay còn lại trong (4.11):

$$W_N^{\left(\frac{N}{4} n_2 + n_3 \right) (k_1 + 2k_2 + 4k_3)}. \quad (4.12)$$

Ta tách hệ số này thành hai phần:

$$W_N^{\left(\frac{N}{4} n_2 + n_3 \right) (k_1 + 2k_2 + 4k_3)} = W_N^{\left(\frac{N}{4} n_2 + n_3 \right) (k_1 + 2k_2)} W_N^{\left(\frac{N}{4} n_2 + n_3 \right) 4k_3}. \quad (4.13)$$

Với thừa số chứa $4k_3$:

$$\begin{aligned} W_N^{\left(\frac{N}{4} n_2 + n_3 \right) 4k_3} &= W_N^{N n_2 k_3} W_N^{4 n_3 k_3} \\ &= W_{N/4}^{n_3 k_3}. \end{aligned} \quad (4.14)$$

Với thừa số còn lại:

$$\begin{aligned} W_N^{\left(\frac{N}{4} n_2 + n_3 \right) (k_1 + 2k_2)} &= W_N^{\frac{N}{4} n_2 (k_1 + 2k_2)} W_N^{n_3 (k_1 + 2k_2)} \\ &= \left(W_N^{N/4} \right)^{n_2 (k_1 + 2k_2)} W_N^{n_3 (k_1 + 2k_2)}. \end{aligned} \quad (4.15)$$

Mà

$$W_N^{N/4} = e^{-j \frac{2\pi}{N} \frac{N}{4}} = e^{-j\pi/2} = -j. \quad (4.16)$$

Suy ra phương trình phân rã hệ số xoay tổng hợp:

$$\boxed{W_N^{\left(\frac{N}{4} n_2 + n_3 \right) (k_1 + 2k_2 + 4k_3)} = (-j)^{n_2 (k_1 + 2k_2)} W_N^{n_3 (k_1 + 2k_2)} W_{N/4}^{n_3 k_3}.} \quad (4.17)$$

Đây là bước quan trọng nhất của Radix-2². Ba thừa số ở vế phải có ba vai trò khác nhau: $(-j)^{n_2 (k_1 + 2k_2)}$ là hệ số xoay tầm thường dùng trong butterfly thứ hai; $W_N^{n_3 (k_1 + 2k_2)}$

là hệ số xoay không tầm thường đặt sau một block butterfly; $W_{N/4}^{n_3 k_3}$ là nhân tử của DFT con chiều dài $N/4$.

4.5 Butterfly thứ hai và phương trình $H(k_1, k_2, n_3)$

Thay (4.17) vào (4.11), ta được

$$\begin{aligned} X[k_1 + 2k_2 + 4k_3] &= \sum_{n_3=0}^{N/4-1} \sum_{n_2=0}^1 \mathcal{B}_N^{k_1} \left(\frac{N}{4} n_2 + n_3 \right) (-j)^{n_2(k_1+2k_2)} \\ &\times W_N^{n_3(k_1+2k_2)} W_{N/4}^{n_3 k_3}. \end{aligned} \quad (4.18)$$

Gom tổng theo n_2 tạo ra butterfly Radix-2 thứ hai. Hàm trung gian $H(k_1, k_2, n_3)$ được viết là

$$\boxed{H(k_1, k_2, n_3) = \underbrace{\underbrace{\left[x[n_3] + (-1)^{k_1} x \left[n_3 + \frac{N}{2} \right] \right]}_{\text{BFI}} + (-j)^{k_1+2k_2} \underbrace{\left[x \left[n_3 + \frac{N}{4} \right] + (-1)^{k_1} x \left[n_3 + \frac{3N}{4} \right] \right]}_{\text{BFI}}}_{\text{BF II}}}. \quad (4.19)$$

Có thể viết ngắn hơn bằng đầu ra butterfly thứ nhất:

$$H(k_1, k_2, n_3) = \mathcal{B}_N^{k_1}(n_3) + (-j)^{k_1+2k_2} \mathcal{B}_N^{k_1} \left(n_3 + \frac{N}{4} \right). \quad (4.20)$$

Khi đó DFT ban đầu được đưa về dạng

$$\boxed{X[k_1 + 2k_2 + 4k_3] = \sum_{n_3=0}^{N/4-1} \left[H(k_1, k_2, n_3) W_N^{n_3(k_1+2k_2)} \right] W_{N/4}^{n_3 k_3}}. \quad (4.21)$$

Phương trình (4.21) cho thấy sau hai butterfly Radix-2, FFT chiều dài N được tách thành bốn DFT con chiều dài $N/4$. Dữ liệu đầu vào của mỗi DFT con là $H(k_1, k_2, n_3)$ sau khi nhân với hệ số $W_N^{n_3(k_1+2k_2)}$. Đây chính là lý do Radix-2² đạt độ phức tạp nhân giống Radix-4 nhưng vẫn giữ cấu trúc butterfly kiểu Radix-2.

4.6 Ý nghĩa phần cứng của các hệ số xoay

Đặt

$$q = k_1 + 2k_2, \quad q \in \{0, 1, 2, 3\}. \quad (4.22)$$

Hệ số xoay nằm trong butterfly thứ hai là

$$C_q = (-j)^q = (-j)^{k_1+2k_2}. \quad (4.23)$$

Bảng 4.1 liệt kê toàn bộ bốn trường hợp của C_q .

Bảng 4.1: Hệ số xoay tầm thường trong cụm Radix-2²

k_1	k_2	$q = k_1 + 2k_2$	$C_q = (-j)^q$	Ý nghĩa phần cứng
0	0	0	1	Giữ nguyên dữ liệu
1	0	1	$-j$	Hoán đổi thực-ảo và đổi dấu phù hợp
0	1	2	-1	Đổi dấu cả phần thực và phần ảo
1	1	3	j	Hoán đổi thực-ảo và đổi dấu phù hợp

Nếu $z = a + jb$, các phép nhân tầm thường được thực hiện như sau:

$$1 \cdot z = a + jb, \quad (4.24)$$

$$(-1) \cdot z = -a - jb, \quad (4.25)$$

$$(-j) \cdot z = b - ja, \quad (4.26)$$

$$j \cdot z = -b + ja. \quad (4.27)$$

Do đó, C_q không cần bộ nhân phức. Trong RTL, phần này có thể được hiện thực bằng bộ ghép kênh (MUX), mạch đảo dấu bù 2 và mạng chuyển mạch hoán đổi phần thực-ảo.

Sau butterfly thứ hai, hệ số xoay còn lại là

$$T_N^{(q)}(n_3) = W_N^{qn_3} = W_N^{n_3(k_1+2k_2)}. \quad (4.28)$$

Với $q = 0$, hệ số này bằng 1 nên không cần nhân. Với $q = 1, 2, 3$, hệ số lần lượt là $W_N^{n_3}$, $W_N^{2n_3}$ và $W_N^{3n_3}$; đây là các vị trí bắt buộc hệ thống phải cấp phát tài nguyên cho một bộ nhân phức đầy đủ.

Điểm cần nhấn mạnh là thứ tự hệ số xoay trong Radix-2² không giống cách trình bày Radix-4 thông thường. Tập hệ số nhân không tầm thường tương đương Radix-4, nhưng chúng xuất hiện sau hai tầng butterfly Radix-2. Cách sắp xếp này tạo ra SFG đều hơn, thuận lợi cho quá trình xếp chồng pipeline và kiến trúc SDF.

4.7 Tóm tắt đặc điểm thuật toán và liên hệ với kiến trúc

Phương trình (4.21) có thể được hiểu như sau: với mỗi $q \in \{0, 1, 2, 3\}$, thuật toán tạo ra một chuỗi mới

$$Y_q(n_3) = H(k_1, k_2, n_3)W_N^{qn_3}, \quad q = k_1 + 2k_2, \quad (4.29)$$

rồi tính DFT chiều dài $N/4$ của chuỗi đó:

$$X[q + 4k_3] = \sum_{n_3=0}^{N/4-1} Y_q(n_3)W_{N/4}^{n_3k_3}. \quad (4.30)$$

Áp dụng lại cùng cách thức cho các DFT con chiều dài $N/4$, rồi $N/16$, v.v., ta thu được thuật toán Radix-2² DIF hoàn chỉnh.

Nếu $N = 4^m$, số cụm Radix-2² theo chiều sâu là $\log_4 N$. Cụm cuối cùng tương ứng DFT chiều dài 4, nên không cần bộ nhân phức không tầm thường. Vì vậy, số bộ nhân phức đầy đủ cần dùng trong kiến trúc pipeline Radix-2² SDF là

$$N_{\text{CMUL}} = \log_4 N - 1. \quad (4.31)$$

Với ví dụ $N = 16$, thuật toán có $\log_4 16 = 2$ cụm Radix-2². Cụm đầu biến FFT-16 thành bốn FFT-4 và có một tầng hệ số xoay không tầm thường. Cụm thứ hai xử lý các FFT-4 nên không cần bộ nhân phức đầy đủ.

Từ góc nhìn thiết kế, Radix-2² có ba đặc điểm quan trọng. Thứ nhất, độ phức tạp nhân phức không tầm thường tương đương Radix-4. Thứ hai, butterfly vẫn là butterfly Radix-2 nên đơn giản hơn Radix-4 đầy đủ. Thứ ba, các hệ số $1, -1, j, -j$ phải được hiện thực bằng logic đổi dấu và hoán đổi phần thực-ảo, không dùng bộ nhân phức. Đây là nền tảng để chương tiếp theo ánh xạ thuật toán sang kiến trúc R2²SDF gồm các tầng phản hồi trễ, butterfly loại I, butterfly loại II, bộ nhớ hệ số xoay và bộ điều khiển đồng bộ.

Chương 5

Thiết kế RTL kiến trúc Radix-2² SDF FFT

Chương này trình bày cách chuyển thuật toán Radix-2² DIF FFT từ mô hình toán học sang kiến trúc RTL có thể hiện thực bằng phần cứng. Trọng tâm của chương là chỉ rõ từng phép toán được ánh xạ thành khối phần cứng nào, dữ liệu đi qua pipeline theo thứ tự nào, phản hồi trễ được điều khiển ra sao, và vì sao kiến trúc R2²SDF có thể giảm số bộ nhân phức nhưng vẫn giữ được đường dữ liệu đều.

5.1 Tham số hệ thống

Trong toàn chương, các thuật ngữ và ký hiệu được dùng thống nhất như sau:

- **N** : Kích thước FFT.
- **DATA_W**: Độ rộng từ dữ liệu số cố định (được đặt tên này để tránh nhầm lẫn với ký hiệu butterfly $\mathcal{B}_N^{k_1}$ và hệ số xoay W_N).
- **D** : Độ dài đường trễ của từng tầng.
- **cnt**: Bộ đếm toàn cục.
- **rot_sel**: Tín hiệu chọn hệ số xoay tầm thường.
- **CMUL**: Bộ nhân phức đầy đủ.

Bảng 5.1: Tham số chính của thiết kế FFT-256 R2²SDF

Tham số	Giá trị	Ghi chú
Kích thước FFT	$N = 256$	Tương ứng 8 tầng Radix-2.
Thuật toán	Radix-2 ² DIF FFT	Hai tầng Radix-2 được gom thành một cụm Radix-2 ² .
Kiến trúc RTL	Single-path Delay Feedback	Một mẫu phức đi vào mỗi chu kỳ xung nhịp.
Thông lượng mục tiêu	1 mẫu phức/chu kỳ xung nhịp sau khi pipeline đầy	Độ trễ ban đầu vẫn tồn tại do các đường trễ.
Thứ tự ngõ vào	Thứ tự tự nhiên	Dữ liệu vào theo thứ tự tự nhiên.
Thứ tự ngõ ra	Thứ tự đảo chữ số / đảo bit	Có thể sắp xếp lại bằng các tầng sau.
Định dạng số	Q1.15, DATA_W = 16 bit	1 bit dấu và 15 bit phần phân số.
Cơ chế scaling	Dịch phải 1 bit sau mỗi butterfly	Giảm rủi ro tràn số.

5.2 Từ phương trình thuật toán sang khối RTL

Ở chương thuật toán, phép biến đổi Radix-2² DIF có thể được nhìn qua biến trung gian $H(k_1, k_2, n_3)$. Biến này gom hai tầng butterfly Radix-2 liên tiếp, trong đó tầng thứ hai chỉ cần nhân với các hệ số xoay tầm thường thuộc tập $\{1, -j, -1, j\}$. Đây chính là điểm giúp phần cứng tránh được một bộ nhân phức ở tầng này.

$$\begin{aligned}
 H(k_1, k_2, n_3) = & \left[x[n_3] + (-1)^{k_1} x \left[n_3 + \frac{N}{2} \right] \right] \\
 & + (-j)^{k_1 + 2k_2} \left[x \left[n_3 + \frac{N}{4} \right] + (-1)^{k_1} x \left[n_3 + \frac{3N}{4} \right] \right].
 \end{aligned} \tag{5.1}$$

Từ (5.1), có thể tách quá trình xử lý thành hai lớp phần cứng. Lớp thứ nhất là các phép cộng/trừ của butterfly Radix-2. Lớp thứ hai là phép xoay tầm thường $(-j)^q$, với $q = k_1 + 2k_2$. Vì q chỉ nhận bốn giá trị 0, 1, 2, 3 nên phép xoay này không cần bộ nhân; chỉ cần đổi dấu và hoán đổi phần thực/phần ảo.

Sau khi tạo $H(k_1, k_2, n_3)$, biểu thức DFT có thể viết lại theo dạng

$$X[k_1 + 2k_2 + 4k_3] = \sum_{n_3=0}^{N/4-1} H(k_1, k_2, n_3) W_N^{n_3(k_1+2k_2)} W_{N/4}^{n_3 k_3}. \quad (5.2)$$

Phần $W_N^{n_3(k_1+2k_2)}$ trong (5.2) là hệ số xoay không tầm thường. Đây là nơi cần bộ nhân phức đầy đủ, vì hệ số có thể là một giá trị phức bất kỳ trên vòng tròn đơn vị. Phần $W_{N/4}^{n_3 k_3}$ thể hiện rằng sau mỗi cụm Radix-2², bài toán còn lại là một DFT ngắn hơn, tiếp tục được xử lý bởi các cụm R2²SDF phía sau.

Bảng 5.2: Ánh xạ từ thành phần thuật toán sang khối RTL

Thành phần toán học	Ý nghĩa	Khối RTL tương ứng
$\mathcal{B}_N^{k_1}(r)$	Butterfly Radix-2 tầng thứ nhất	BF2I trong tầng SDF
$(-j)^q, q = k_1 + 2k_2$	Hệ số xoay tầm thường	<code>trivial_rotator</code> trong BF2II
$H(k_1, k_2, n_3)$	Kết quả sau hai tầng butterfly Radix-2	Ngõ ra của một cặp BF2I + BF2II
$W_N^{n_3(k_1+2k_2)}$	Twiddle không tầm thường sau cụm Radix-2 ²	CMUL + bộ nhớ hệ số xoay
$W_{N/4}^{n_3 k_3}$	DFT con chiều dài $N/4$	Các cụm R2 ² SDF tiếp theo
Ánh xạ chỉ số n, k	Quyết định mẫu nào được ghép butterfly với nhau	Đường trễ + <code>cnt</code> + tín hiệu chọn pha

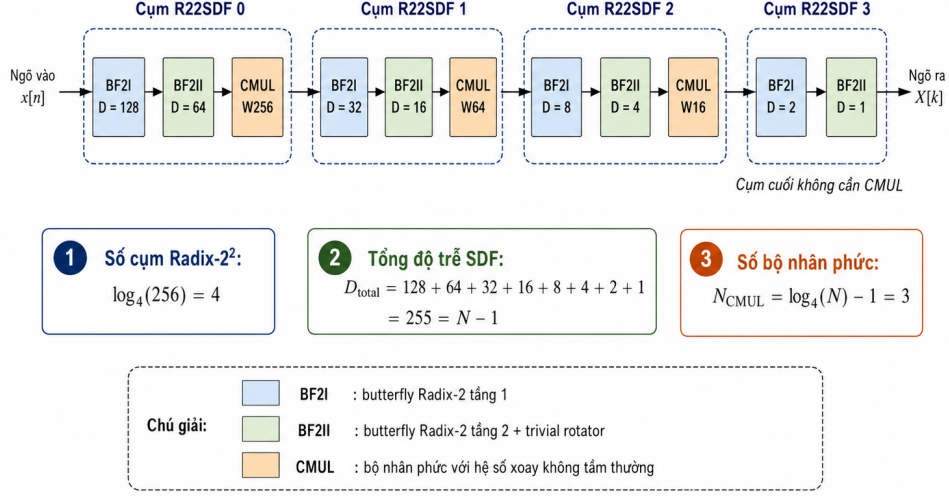
5.3 Sơ đồ khối tổng thể FFT-256 R2²SDF

Với FFT-256, số tầng Radix-2 thông thường là $\log_2(256) = 8$. Trong kiến trúc R2²SDF, hai tầng Radix-2 được gom thành một cụm Radix-2², nên toàn bộ bộ xử lý gồm $\log_4(256) = 4$ cụm. Mỗi cụm có một BF2I, một BF2II và một bộ nhân phức đầy đủ nếu cụm đó chưa phải là cụm cuối.

Luồng xử lý tổng quát có thể viết gọn như sau:

Ngõ vào $\rightarrow$ BF2I($D = 128$) $\rightarrow$ BF2II($D = 64$) $\rightarrow$ CMUL(W_{256}) $\rightarrow$ BF2I($D = 32$) $\rightarrow$ BF2II($D = 16$) $\rightarrow$ CMUL(W_{64}) $\rightarrow$ BF2I($D = 8$) $\rightarrow$ BF2II($D = 4$) $\rightarrow$ CMUL(W_{16}) $\rightarrow$ BF2I($D = 2$) $\rightarrow$ BF2II($D = 1$) $\rightarrow$ Ngõ ra

5.3 Sơ đồ khối tổng thể FFT-256 R22SDF



Hình 5.1: Sơ đồ khối top level FFT-256 R22SDF

Hình 5.1: Sơ đồ khối top level.

Các đường trễ giảm theo lũy thừa của 2. Đây là đặc trưng của FFT dạng DIF: ở các tầng đầu, hai mẫu cần ghép butterfly cách nhau xa; càng về sau, khoảng cách ghép mẫu càng ngắn.

$$D = \{128, 64, 32, 16, 8, 4, 2, 1\}, \quad (5.3)$$

$$D_{\text{total}} = 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255 = N - 1, \quad (5.4)$$

$$N_{\text{CMUL}} = \log_4(N) - 1 = \log_4(256) - 1 = 3. \quad (5.5)$$

Như vậy, thay vì cần bộ nhân phức ở nhiều tầng Radix-2, thiết kế FFT-256 R2²SDF chỉ cần ba CMUL đầy đủ. Cụm cuối không cần CMUL vì phần DFT còn lại có chiều dài 4, các hệ số tương ứng chỉ là các hệ số tầm thường có thể xử lý bằng BF2II.

5.4 Nguyên lý tầng SDF

SDF là viết tắt của Single-path Delay Feedback. “Một đường dữ liệu” nghĩa là dữ liệu chỉ đi trên một luồng chính: mỗi chu kỳ xung nhịp nhận một mẫu phức, không tách thành nhiều đường song song. “Phản hồi trễ” nghĩa là một phần kết quả trung gian được đưa ngược vào đường trễ để chờ ghép với mẫu đến sau.

Một tầng SDF có độ dài D hoạt động theo chu kỳ $2D$ chu kỳ xung nhịp. Trong nửa chu kỳ đầu, tầng chủ yếu nạp mẫu vào đường trễ. Trong nửa chu kỳ sau, mẫu mới

được ghép với mẫu cũ đã được giữ lại đúng D chu kỳ xung nhịp để tạo phép butterfly. Hai kết quả của butterfly không đi ra cùng lúc; chúng được phân kênh theo thời gian. Một nhánh đi tiếp trong đường dữ liệu, nhánh còn lại được đưa vào phản hồi để xuất hiện ở thời điểm phù hợp.

Điểm cần nắm chắc là đường trễ không phải bộ nhớ phụ “lưu tạm cho tiện”. Nó là phần quyết định đúng cặp mẫu butterfly. Nếu D sai, hoặc tín hiệu chọn pha nạp/tính lệch một chu kỳ xung nhịp, kết quả FFT sẽ sai hoàn toàn.

5.5 Thiết kế BF2I – butterfly Radix-2

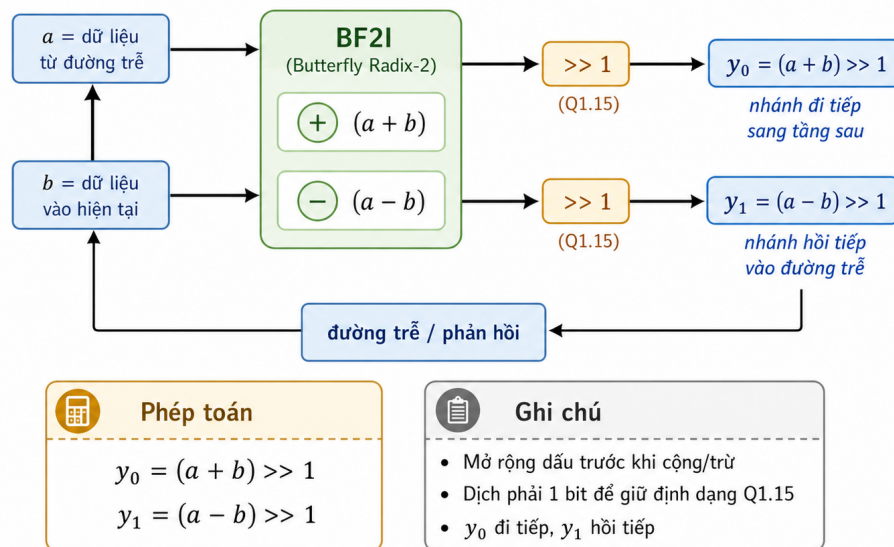
BF2I hiện thực tầng butterfly Radix-2 đầu tiên trong mỗi cụm Radix-2². Khối này không sử dụng hệ số xoay. Nhiệm vụ của nó là cộng và trừ hai mẫu phức cách nhau D chu kỳ xung nhịp: mẫu cũ a lấy từ đường trễ và mẫu hiện tại b đi vào tầng.

Vì dữ liệu dùng định dạng Q1.15, phép cộng hai số cùng biên độ có thể làm tăng bit. Để giữ độ rộng dữ liệu cố định DATA_W = 16 bit, phép cộng/trừ được mở rộng dấu trước khi tính, sau đó dịch phải 1 bit để đưa kết quả về lại miền Q1.15. Với mỗi phần thực và phần ảo, khối BF2I tính:

$$y_0 = (a + b) \gg 1, \quad (5.6)$$

$$y_1 = (a - b) \gg 1. \quad (5.7)$$

5.5 Thiết kế BF2I – butterfly Radix-2



Hình 5.2: Sơ đồ khối BF2I

Hình 5.2: Sơ đồ khối BF2I.

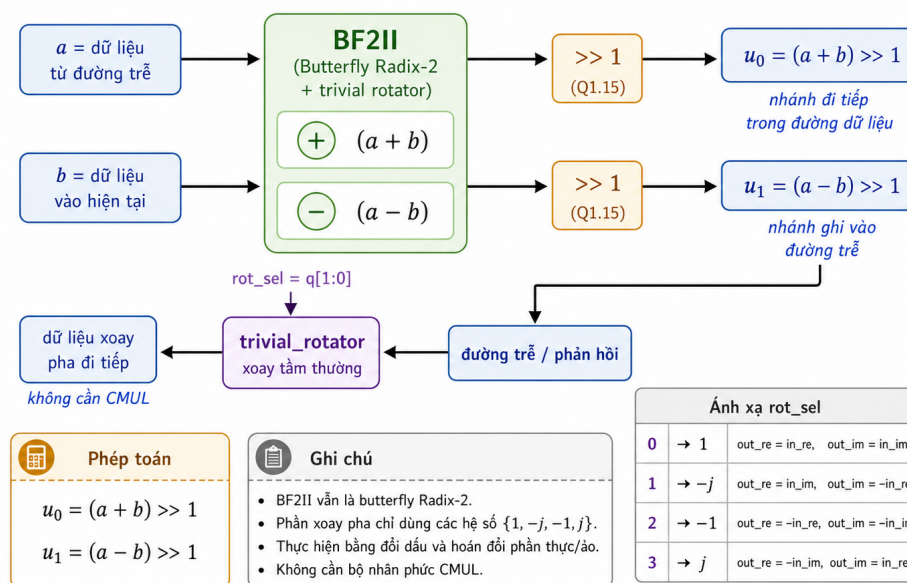
5.6 Thiết kế BF2II và bộ xoay tầm thường

Tương tự BF2I, dữ liệu sau công/trừ được chia tỷ lệ để giảm rủi ro tràn số:

$$u_0 = (a + b) \gg 1, \quad (5.8)$$

$$u_1 = (a - b) \gg 1. \quad (5.9)$$

5.6 Thiết kế BF2II và bộ xoay tầm thường



Hình 5.3: Sơ đồ khối BF2II và bộ xoay tầm thường

Hình 5.3: Sơ đồ khối BF2II.

Do SDF là kiến trúc phân kênh theo thời gian, ngõ ra của BF2II không nên hiểu như một phương trình tính duy nhất. Ở pha tính, u_0 đi tiếp trong đường dữ liệu, còn u_1 được ghi vào đường trễ. Ở pha còn lại, dữ liệu u_1 đã trễ đủ D chu kỳ xung nhịp được đọc ra và đi qua `trivial_rotator` trước khi đi tiếp. Nói ngắn gọn: BF2II vừa tính butterfly, vừa sắp lịch xuất hai nhánh kết quả theo thời gian.

Tín hiệu chọn xoay được ký hiệu là `rot_sel` = $q[1 : 0]$, với $q = k_1 + 2k_2$. Bảng 5.3 trình bày ánh xạ phần cứng của `trivial_rotator`.

Bảng 5.3: Ánh xạ hệ số xoay tầm thường trong BF2II

<code>rot_sel</code> / q	$C_q = (-j)^q$	Ánh xạ phần thực	Ánh xạ phần ảo
0	1	$out_re = in_re$	$out_im = in_im$
1	$-j$	$out_re = in_im$	$out_im = -in_re$
2	-1	$out_re = -in_re$	$out_im = -in_im$
3	j	$out_re = -in_im$	$out_im = in_re$

Bảng trên cũng cho thấy vì sao phép xoay này rẻ về phần cứng: không có phép nhân nào xuất hiện. Khối chỉ cần MUX, dây nối và logic đảo dấu.

5.7 Thiết kế bộ nhân phức CMUL

Sau mỗi cụm BF2I + BF2II, ngoại trừ cụm cuối, dữ liệu phải nhân với hệ số xoay không tầm thường. Khối này được gọi là CMUL. Chương này quy định $z = z_{re} + jz_{im}$ cho dữ liệu vào và $t = t_{re} + jt_{im}$ cho hệ số xoay đọc từ ROM.

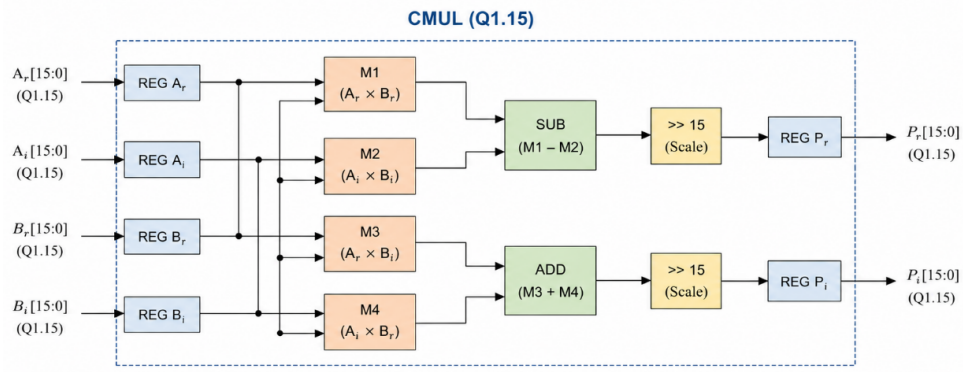
$$z \cdot t = (z_{re} + jz_{im})(t_{re} + jt_{im}), \quad (5.10)$$

$$y_{re} = z_{re}t_{re} - z_{im}t_{im}, \quad (5.11)$$

$$y_{im} = z_{re}t_{im} + z_{im}t_{re}. \quad (5.12)$$

Cách triển khai trực tiếp cần bốn bộ nhân thực và hai bộ cộng/trừ thực. Trên FPGA, bốn bộ nhân này thường ánh xạ vào DSP slice. Trên ASIC, đây là một trong các khối ảnh hưởng mạnh đến diện tích phần cứng, công suất và định thời. Vì vậy, việc giảm số CMUL xuống còn $\log_4(N) - 1$ là lợi ích quan trọng nhất của kiến trúc R2²SDF.

Với số cố định Q1.15, tích của hai số 16 bit tạo ra kết quả trung gian rộng hơn 16 bit. Do đó, CMUL cần có bước cắt bit hoặc làm tròn về Q1.15. Nếu chỉ cắt bớt bit, phần cứng đơn giản hơn nhưng nhiều lượng tử lớn hơn. Nếu làm tròn, kết quả đẹp hơn nhưng phức tạp hơn.



Hình 5.4: Sơ đồ khối CMUL.

5.8 Bộ nhớ hệ số xoay và sinh địa chỉ

Bộ nhớ hệ số xoay lưu các hệ số W_M^r ở dạng số cố định Q1.15. Vì pipeline xử lý một mẫu mỗi chu kỳ xung nhịp, cách dễ triển khai và dễ đồng bộ thời nhất là dùng một ROM riêng cho từng tầng CMUL. Với FFT-256, có ba CMUL nên cần ba nhóm hệ số chính: W_{256} , W_{64} và W_{16} .

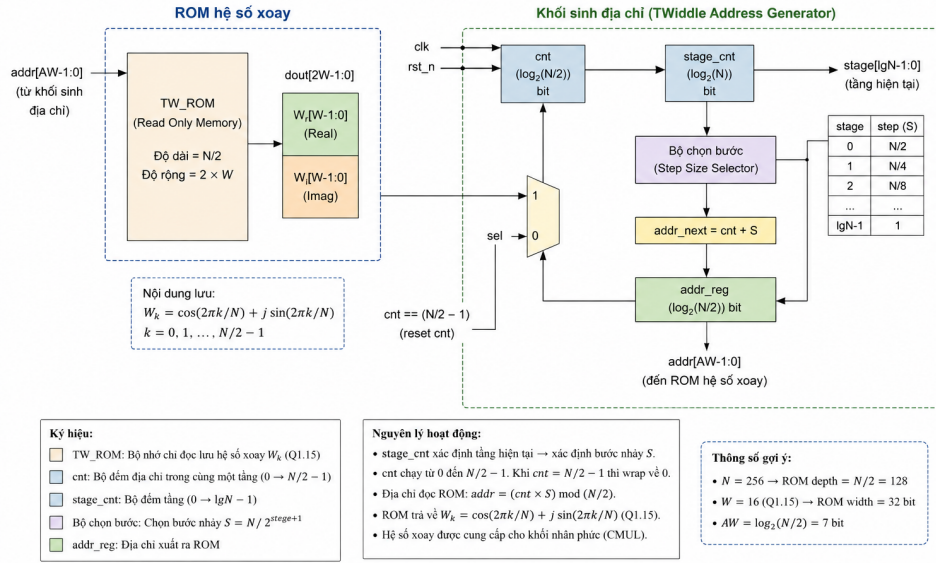
$$T_M(q, r) = W_M^{qr}, \quad q \in \{0, 1, 2, 3\}. \quad (5.13)$$

Bảng 5.4: Bộ nhớ hệ số xoay theo từng tầng CMUL

Tầng CMUL	Sau cụm	Chiều dài DFT hiện tại M	Hệ số cần lưu	Ghi chú
0	Cụm 0	256	W_{256}^{qr}	$q = 0$ có thể bỏ qua vì hệ số bằng 1.
1	Cụm 1	64	W_{64}^{qr}	Địa chỉ ROM ngắn hơn tầng 0.
2	Cụm 2	16	W_{16}^{qr}	Nhiều hệ số rơi vào trường hợp tầm thường.
–	Cụm 3	4	Không cần ROM	Cụm cuối chỉ còn hệ số tầm thường.

Trong RTL, địa chỉ ROM không nên sinh bằng phép chia hoặc modulo phức tạp vì các phép này không thân thiện với định thời. Cách hợp lý hơn là dùng `cnt` đồng bộ toàn cục và lấy các bit tương ứng cho từng tầng. Hai bit cao của từng cụm xác định q hoặc `rot_sel`, còn các bit thấp hơn tạo r , tức địa chỉ đọc ROM.

5.8 Bộ nhớ hệ số xoay và sinh địa chỉ



Hình 5.5: Sơ đồ khối bộ nhớ hệ số xoay.

5.9 Đường trễ và bộ nhớ phản hồi

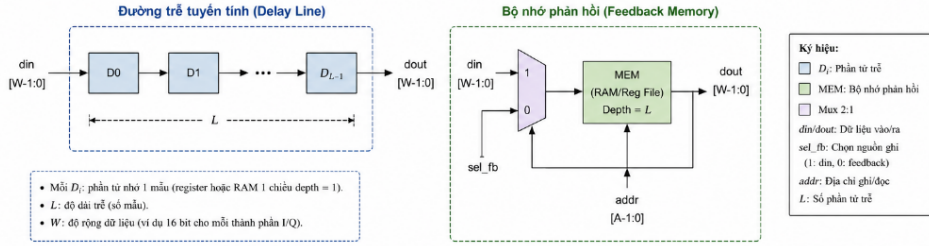
Trong R2²SDF, đường trễ là phần giữ mẫu cũ để ghép với mẫu mới ở đúng khoảng cách D . Mỗi tầng SDF cần một đường trễ có độ dài D mẫu phức. Với FFT-256, các giá trị D lần lượt là 128, 64, 32, 16, 8, 4, 2 và 1. Tổng số ô nhớ phức cần dùng là:

$$D_{\text{total}} = \sum_i D_i = 2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = 255 = N - 1. \quad (5.14)$$

Trong đồ án, đường trễ được hiện thực bằng thanh ghi dịch. Cách này tốn nhiều thanh ghi hơn so với RAM, nhưng rất dễ quan sát trên dạng sóng: dữ liệu đi qua từng chu kỳ xung nhịp và xuất hiện ở ngõ ra sau đúng D chu kỳ.

Sau khi thiết kế đã đúng chức năng, các đường trễ dài như $D = 128, 64, 32$ và 16 có thể chuyển sang RAM vòng hoặc BRAM để giảm tài nguyên logic. Các đường trễ ngắn như $D = 8, 4, 2$ và 1 vẫn có thể giữ dạng thanh ghi dịch vì chi phí nhỏ và định thời thường tốt hơn.

5.9 Đường trễ và bộ nhớ phản hồi

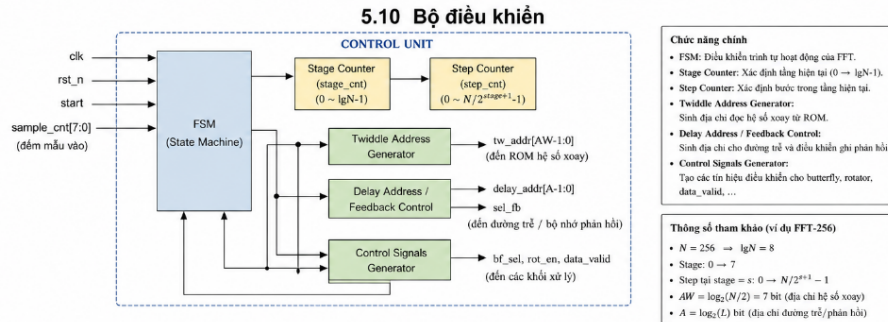


Hình 5.6: Sơ đồ khối đường trễ.

5.10 Bộ điều khiển

Một ưu điểm của R2²SDF là bộ điều khiển không cần FSM lớn. Do lịch xử lý có tính tuần hoàn, một bộ đếm toàn cục cnt có độ rộng $\log_2(N)$ bit có thể dùng đồng thời cho ba việc: chọn pha nạp/tính của từng tầng trễ, tạo rot_sel cho BF2II và sinh địa chỉ đọc bộ nhớ hệ số xoay.

Với FFT-256, cnt có 8 bit, ký hiệu cnt [7:0]. Mỗi tầng trễ lấy một bit của cnt để xác định pha hiện tại. Các cặp bit lân cận được dùng làm rot_sel cho BF2II hoặc làm một phần địa chỉ ROM. Bảng 5.5 trình bày cách dùng bit điều khiển theo từng tầng.



Hình 5.7: Sơ đồ khối bộ điều khiển.

Bảng 5.5: Ánh xạ bit điều khiển từ bộ đếm toàn cục `cnt`

Tầng delay D	Bit <code>cnt</code> dùng cho pha	Hai bit điều khiển <code>rot_sel</code>	Khối sử dụng
128	<code>cnt[7]</code>	<code>cnt[7:6]</code>	BF2I cụm 0
64	<code>cnt[6]</code>	<code>cnt[7:6]</code>	BF2II cụm 0
32	<code>cnt[5]</code>	<code>cnt[5:4]</code>	BF2I cụm 1
16	<code>cnt[4]</code>	<code>cnt[5:4]</code>	BF2II cụm 1
8	<code>cnt[3]</code>	<code>cnt[3:2]</code>	BF2I cụm 2
4	<code>cnt[2]</code>	<code>cnt[3:2]</code>	BF2II cụm 2
2	<code>cnt[1]</code>	<code>cnt[1:0]</code>	BF2I cụm 3
1	<code>cnt[0]</code>	<code>cnt[1:0]</code>	BF2II cụm 3

Bảng này được xem là lịch điều khiển logic ở mức kiến trúc. Khi viết RTL, cần kiểm tra lại quy ước đếm bắt đầu từ 0 hay từ 1, cạnh reset, và thời điểm `valid_in` bắt đầu. Chỉ cần lệch một chu kỳ xung nhịp giữa `cnt` và dữ liệu, toàn bộ lịch phản hồi trễ sẽ sai.

Vì vậy, bộ điều khiển nên đi kèm tín hiệu `valid_pipe` để đánh dấu dữ liệu hợp lệ sau khi pipeline đầy. Ngõ ra không nên được xem là hợp lệ ngay từ chu kỳ xung nhịp đầu tiên, vì các đường trễ cần thời gian nạp trạng thái ban đầu.

5.11 Ước lượng tài nguyên

Bảng 5.6 tóm tắt tài nguyên chính của thiết kế FFT-256 R2²SDF. Đây là ước lượng ở mức kiến trúc, dùng để định hướng thiết kế RTL trước khi tổng hợp. Con số thực tế sau tổng hợp sẽ phụ thuộc vào cách viết RTL, cách ánh xạ đường trễ, cách làm tròn số cố định và công nghệ mục tiêu.

Bảng 5.6: Ước lượng tài nguyên cho FFT-256 R2²SDF

Thành phần	Số lượng cho FFT-256	Nhận xét
BF2I	4	Một khối cho mỗi cụm Radix-2 ² .
BF2II	4	Tích hợp <code>trivial_rotator</code> .
CMUL	3	Tương ứng $\log_4(256) - 1$.
Thanh ghi trễ phức	255 mẫu phức	Tương ứng tổng $D = N - 1$ nếu dùng thanh ghi dịch.
bộ nhớ hệ số xoay	3	Lần lượt cho W_{256} , W_{64} và W_{16} .
Bộ đếm toàn cục	8 bit	Vì $\log_2(256) = 8$.

Nếu mỗi mẫu phức gồm hai phần Q1.15, một mẫu phức cần 32 bit. Vì vậy, riêng đường trễ dạng thanh ghi dịch cần khoảng

$$255 \times 32 = 8160 \text{ bit} \quad (5.15)$$

thanh ghi. Con số này chưa lớn ở mức mô phỏng và kiểm chứng, nhưng trên FPGA/ASIC vẫn nên tối ưu ở các đường trễ dài nếu thiết kế cần tiết kiệm tài nguyên.

5.12 Tổng kết chương

Chương này đã chuyển phần thuật toán Radix-2² DIF FFT sang kiến trúc RTL cụ thể cho FFT-256. Các khối chính của đường dữ liệu gồm BF2I, BF2II, `trivial_rotator`, CMUL, bộ nhớ hệ số xoay, đường trễ và bộ điều khiển `cnt`. Điểm cốt lõi của thiết kế là giữ luồng dữ liệu một mẫu phức mỗi chu kỳ xung nhịp, dùng phản hồi trễ để ghép đúng cặp butterfly, và chỉ dùng CMUL ở các vị trí thật sự cần hệ số xoay không tầm thường.

Trong chương tiếp theo, RTL được xây từ dưới lên: kiểm chứng BF2I và BF2II trước, sau đó kiểm chứng `trivial_rotator`, CMUL, bộ nhớ hệ số xoay, từng tầng SDF, rồi mới ghép toàn bộ thành FFT 256 điểm hoàn chỉnh.

Chương 6

Triển khai RTL và kiểm chứng chức năng hệ thống FFT-256 R2²SDF

6.1 Mục tiêu và phạm vi chương

Chương 5 đã trình bày cách ánh xạ thuật toán Radix-2² DIF FFT sang kiến trúc R2²SDF ở mức thiết kế RTL. Từ nền tảng đó, chương này mô tả quá trình hiện thực kiến trúc bằng ngôn ngữ SystemVerilog, cách ghép các khối nhỏ thành bộ xử lý FFT-16 và FFT-256, và cách kiểm tra kết quả RTL bằng mô phỏng chức năng so với mô hình đối chiếu viết bằng Python.

Mục tiêu của chương không phải là lặp lại phần chứng minh thuật toán. Trọng tâm ở đây là chỉ rõ: mỗi phép toán trong kiến trúc được đặt vào mô-đun phần cứng nào, dữ liệu đi qua các tầng theo lịch nào, các tín hiệu điều khiển được căn thời gian ra sao, và kết quả mô phỏng cho thấy thiết kế đúng đến mức nào.

Ba câu hỏi chính của chương là:

1. Các khối trong sơ đồ R2²SDF được hiện thực thành những mô-đun RTL nào?
2. Các tín hiệu `valid`, `sync`, `phase_sel`, `rot_sel` và `tw_addr` được căn theo dòng dữ liệu như thế nào?
3. Kết quả kiểm chứng chức năng cho thấy thiết kế đã đạt được gì, và phần nào vẫn chưa thể kết luận khi chưa tổng hợp phần cứng?

Vì vậy, chương này là cầu nối giữa công thức toán học ở Chương 4, kiến trúc phần cứng ở Chương 5 và sản phẩm RTL hoàn chỉnh của đề án.

6.2 Quy ước giao tiếp RTL và định dạng dữ liệu

Các mô-đun cấp cao dùng giao tiếp dữ liệu theo luồng ở dạng tối giản. Thiết kế không dùng giao thức bắt tay đầy đủ **ready/valid**, vì không có tín hiệu **ready** phản hồi từ khối phía sau. Hệ thống được giả thiết luôn có thể nhận mẫu mới khi **valid_in** ở mức cao. Khi **valid_in** xuống thấp, bộ đếm và các thanh ghi của đường ống giữ nguyên trạng thái, nhờ đó lịch điều khiển không bị trượt so với dữ liệu.

```
1 module fft256_r22sdf_top #(
2     parameter int DATA_W = 16
3 )(
4     input  logic          clk,
5     input  logic          rst_n,
6     input  logic          valid_in,
7     input  logic          sync_in,
8     input  logic signed [DATA_W-1:0] din_re,
9     input  logic signed [DATA_W-1:0] din_im,
10    output logic          valid_out,
11    output logic          sync_out,
12    output logic signed [DATA_W-1:0] dout_re,
13    output logic signed [DATA_W-1:0] dout_im
14 );
```

Listing 6.1: Giao diện cấp cao của `fft256_r22sdf_top.sv`

Trong giao diện trên, **valid_in** cho biết mẫu đầu vào hiện tại có hợp lệ hay không. Có thể xem tín hiệu này như tín hiệu cho phép cập nhật ở mức dữ liệu. Tín hiệu **sync_in** đánh dấu mẫu đầu tiên của một khung dữ liệu và được dùng để đưa bộ đếm cục bộ về pha khởi đầu. Tín hiệu **rst_n** là tín hiệu đặt lại toàn cục mức thấp, chủ yếu dùng khi khởi động hệ thống.

Dữ liệu vào và ra dùng định dạng số cố định Q1.15 có dấu. Mỗi phần thực và phần ảo có độ rộng 16 bit, do đó một mẫu phức chiếm 32 bit. Sau mỗi phép cộng/trừ trong butterfly, kết quả được dịch phải một bit để giảm nguy cơ tràn số. Đối với FFT-256, có 8 tầng butterfly Radix-2, nên hệ số chia tỷ lệ lý tưởng của toàn bộ bộ xử lý là

$$G = 2^{-8} = \frac{1}{256}. \quad (6.1)$$

Khi so sánh RTL với mô hình Python hoặc với FFT tính bằng số thực dấu phẩy động, hệ số chia tỷ lệ này bắt buộc phải được đưa vào. Nếu không, kết quả có thể đúng về vị trí bin tần số nhưng sai về biên độ.

6.3 Các khối RTL cơ bản

6.3.1 Khối xoay pha tầm thường trivial_rotator.sv

Khối trivial_rotator thực hiện phép nhân với các hệ số 1 , $-j$, -1 và j . Đây là điểm quan trọng của thuật toán Radix-2²: các hệ số xoay tầm thường không cần bộ nhân phức, mà chỉ cần đổi dấu và hoán đổi phần thực-ảo.

```
1  always_comb begin
2      unique case (rot_sel)
3          2'd0: begin                // nhân với 1
4              out_re = in_re;
5              out_im = in_im;
6          end
7          2'd1: begin                // nhân với -j
8              out_re = in_im;
9              out_im = -in_re;
10         end
11         2'd2: begin                // nhân với -1
12             out_re = -in_re;
13             out_im = -in_im;
14         end
15         2'd3: begin                // nhân với j
16             out_re = -in_im;
17             out_im = in_re;
18         end
19     endcase
20 end
```

Listing 6.2: Lỗi tổ hợp của trivial_rotator.sv

Khối này được giữ ở dạng mạch tổ hợp. Nếu chèn thêm thanh ghi vào khối xoay pha mà không bù lại độ trễ điều khiển, tín hiệu `rot_sel` có thể lệch một chu kỳ so với dữ liệu. Khi đó phép xoay pha vẫn đúng về mặt mạch con, nhưng lại áp dụng nhầm vào mẫu dữ liệu khác.

6.3.2 Butterfly Radix-2 butterfly_radix2_scaled.sv

Khối butterfly nhận hai mẫu phức a và b , sau đó tạo ra nhánh tổng và nhánh hiệu. Trước khi cộng hoặc trừ, mỗi toán hạng 16 bit được mở rộng dấu lên 17 bit. Sau phép cộng/trừ, kết quả được dịch phải số học một bit để đưa về lại định dạng Q1.15.

```
1  logic signed [DATA_W:0] sum_re, sum_im;
2  logic signed [DATA_W:0] dif_re, dif_im;
3
4  assign sum_re = {a_re[DATA_W-1], a_re} + {b_re[DATA_W-1], b_re};
```

```

5 assign sum_im = {a_im[DATA_W-1], a_im} + {b_im[DATA_W-1], b_im};
6 assign dif_re = {a_re[DATA_W-1], a_re} - {b_re[DATA_W-1], b_re};
7 assign dif_im = {a_im[DATA_W-1], a_im} - {b_im[DATA_W-1], b_im};
8
9 assign y0_re = sum_re >>> 1;
10 assign y0_im = sum_im >>> 1;
11 assign y1_re = dif_re >>> 1;
12 assign y1_im = dif_im >>> 1;

```

Listing 6.3: Tính tổng và hiệu có chia tỷ lệ trong butterfly Radix-2

Cách chia tỷ lệ này giúp giảm rủi ro tràn số trong biểu diễn số cố định. Đổi lại, biên độ đầu ra bị giảm sau mỗi tầng.

6.3.3 Bộ nhân phức Q1.15 complex_multiplier_q15.sv

Bộ nhân phức đầy đủ chỉ được dùng tại các vị trí cần nhân với hệ số xoay không tầm thường. Với dữ liệu vào $z = z_{re} + jz_{im}$ và hệ số xoay $t = t_{re} + jt_{im}$, kết quả được tính theo

$$y_{re} = z_{re}t_{re} - z_{im}t_{im}, \quad (6.2)$$

$$y_{im} = z_{re}t_{im} + z_{im}t_{re}. \quad (6.3)$$

Trong định dạng Q1.15, tích của hai số 16 bit tạo ra kết quả trung gian có dạng Q2.30. Sau đó kết quả được dịch về lại Q1.15. Bản RTL chốt ngõ ra của bộ nhân phức thêm một chu kỳ để giảm độ dài đường tổ hợp qua các phép nhân và phép cộng/trừ.

```

1 logic signed [2*DATA_W-1:0] p_re_re, p_im_im;
2 logic signed [2*DATA_W-1:0] p_re_im, p_im_re;
3 logic signed [2*DATA_W :0] acc_re, acc_im;
4
5 assign p_re_re = z_re * tw_re;
6 assign p_im_im = z_im * tw_im;
7 assign p_re_im = z_re * tw_im;
8 assign p_im_re = z_im * tw_re;
9
10 assign acc_re = p_re_re - p_im_im;
11 assign acc_im = p_re_im + p_im_re;
12
13 always_ff @(posedge clk or negedge rst_n) begin
14     if (!rst_n) begin
15         y_re      <= '0;
16         y_im      <= '0;
17         valid_out <= 1'b0;

```

```

18         sync_out <= 1'b0;
19     end else begin
20         valid_out <= valid_in;
21         sync_out <= sync_in;
22         y_re      <= acc_re >>> 15;
23         y_im      <= acc_im >>> 15;
24     end
25 end

```

Listing 6.4: Nhân phức Q1.15 và chốt ngõ ra một chu kỳ

6.3.4 Đường trễ tuyến tính và đường trễ phản hồi

Thiết kế dùng hai dạng đường trễ. Khối `delay_line_shift` là thanh ghi dịch tuyến tính cho các trường hợp trễ thông thường. Riêng trong vòng phản hồi SDF, thiết kế dùng `sdf_feedback_delay` để tránh lệch một chu kỳ tại ranh giới giữa hai pha nạp và tính.

```

1  logic signed [DATA_W-1:0] sr_re [0:DELAY-1];
2  logic signed [DATA_W-1:0] sr_im [0:DELAY-1];
3
4  assign dout_re = sr_re[DELAY-1];
5  assign dout_im = sr_im[DELAY-1];
6
7  always_ff @(posedge clk or negedge rst_n) begin
8      if (!rst_n) begin
9          for (int i = 0; i < DELAY; i++) begin
10             sr_re[i] <= '0;
11             sr_im[i] <= '0;
12         end
13     end else if (valid_in) begin
14         sr_re[0] <= din_re;
15         sr_im[0] <= din_im;
16         for (int i = 1; i < DELAY; i++) begin
17             sr_re[i] <= sr_re[i-1];
18             sr_im[i] <= sr_im[i-1];
19         end
20     end
21 end

```

Listing 6.5: Mảng thanh ghi dịch có ngõ ra tổ hợp trong `sdf_feedback_delay.sv`

Điểm cần giữ chặt là dữ liệu, `valid` và `sync` phải có cùng độ trễ hiệu dụng. Nếu dữ liệu đã đúng nhưng cờ hợp lệ hoặc cờ đầu khung bị lệch, testbench sẽ đọc sai vị trí mẫu, làm kết quả so sánh bị sai dù phần tính toán bên trong không hỏng.

6.3.5 Bộ nhớ hệ số xoay

Bộ nhớ hệ số xoay lưu các giá trị W_M^k đã được lượng tử hóa sang Q1.15. Với FFT-256, ba nhóm hệ số chính là `tw256.hex`, `tw64.hex` và `tw16.hex`. Bộ nhớ được đọc theo kiểu tổ hợp; độ trễ một chu kỳ nằm ở bộ nhân phức phía sau.

```
1 logic [2*DATA_W-1:0] rom [0:(1<<ADDR_W)-1];
2 logic [2*DATA_W-1:0] word;
3
4 initial begin
5     $readmemh(ROM_FILE, rom);
6 end
7
8 assign word = rom[addr];
9 assign tw_re = word[2*DATA_W-1:DATA_W];
10 assign tw_im = word[DATA_W-1:0];
```

Listing 6.6: Cấu trúc đọc bộ nhớ hệ số xoay

Các hệ số ban đầu được tính ở dạng số thực, sau đó nhân với 2^{15} và đổi sang số nguyên có dấu 16 bit.

6.4 Tầng SDF và khối Radix- 2^2 SDF

6.4.1 Tầng BF2I

Tầng BF2I điều khiển hai pha của một tầng SDF. Khi `phase_sel=0`, tầng ở pha nạp: dữ liệu đầu vào được ghi vào đường trễ, còn dữ liệu đầu ra là mẫu đã được giữ trong đường trễ từ trước. Khi `phase_sel=1`, tầng ở pha tính: mẫu hiện tại được ghép với mẫu lấy từ đường trễ để thực hiện butterfly.

```
1 always_comb begin
2     if (!phase_sel) begin
3         delay_in_re = din_re;
4         delay_in_im = din_im;
5         dout_re     = delay_out_re;
6         dout_im     = delay_out_im;
7     end else begin
8         a_re        = delay_out_re;
9         a_im        = delay_out_im;
10        b_re        = din_re;
11        b_im        = din_im;
12        delay_in_re = y1_re;
13        delay_in_im = y1_im;
14        dout_re     = y0_re;
15        dout_im     = y0_im;
```

```

16     end
17 end

```

Listing 6.7: Mạch chọn dữ liệu chính trong tầng BF2I

Trong pha tính, nhánh tổng y_0 đi tiếp sang tầng sau, còn nhánh hiệu y_1 được đưa trở lại đường trễ để xuất hiện đúng thời điểm ở nửa chu kỳ kế tiếp.

6.4.2 Tầng BF2II và trivial_rotator

Tầng BF2II có cấu trúc gần giống BF2I, nhưng có thêm phép xoay pha tầm thường. Khi dữ liệu trễ đi ra ở pha phù hợp, nó được đưa qua `trivial_rotator` trước khi đi tiếp. Điều này tương ứng với hệ số $(-j)^q$ trong thuật toán Radix-2².

```

1 trivial_rotator #(
2     .DATA_W(DATA_W)
3 ) u_rot (
4     .rot_sel(rot_sel),
5     .in_re  (delay_out_re),
6     .in_im  (delay_out_im),
7     .out_re (rot_re),
8     .out_im (rot_im)
9 );
10
11 always_comb begin
12     if (!phase_sel) begin
13         delay_in_re = din_re;
14         delay_in_im = din_im;
15         dout_re     = rot_re;
16         dout_im     = rot_im;
17     end else begin
18         a_re        = delay_out_re;
19         a_im        = delay_out_im;
20         b_re        = din_re;
21         b_im        = din_im;
22         delay_in_re = y1_re;
23         delay_in_im = y1_im;
24         dout_re     = y0_re;
25         dout_im     = y0_im;
26     end
27 end

```

Listing 6.8: Vị trí khối xoay pha tầm thường trong tầng BF2II

Vị trí đặt khối xoay pha phải đi cùng lịch điều khiển. Nếu `rot_sel` sinh đúng giá trị nhưng lệch thời điểm, kết quả vẫn sai. Đây là lý do quá trình kiểm chứng phải quan sát cả dữ liệu lẫn tín hiệu điều khiển trên dạng sóng.

6.4.3 Khối Radix-2²SDF có điều khiển cục bộ

Mỗi khối `r22sdf_block_controlled` gồm một tầng BF2I, một tầng BF2II và tùy chọn một bộ nhân phức. Với FFT-256, ba khối đầu có bộ nhân phức, còn khối cuối không cần vì phần DFT còn lại chỉ dài 4 điểm và các hệ số tương ứng là tầm thường.

Khối điều khiển dùng bộ đếm cục bộ thay vì chỉ dựa vào một bộ đếm toàn cục. Khi `sync_in` xuất hiện cùng `valid_in`, bộ đếm được đưa về 0 ngay trong chu kỳ đang xét. Cách làm này giúp mẫu đầu tiên của khung nhận đúng mã điều khiển, tránh lệch pha một chu kỳ.

```
1 assign current_cnt = (valid_in && sync_in) ? '0 : cnt;
2
3 always_ff @(posedge clk or negedge rst_n) begin
4     if (!rst_n)
5         cnt <= '0;
6     else if (valid_in) begin
7         if (sync_in)
8             cnt <= CNT_W'(1);
9         else
10            cnt <= cnt + 1'b1;
11    end
12 end
```

Listing 6.9: Bộ đếm cục bộ và giá trị đếm hiện hành

Từ giá trị `current_cnt`, khối sinh ra tín hiệu chọn pha cho hai tầng con và tín hiệu chọn xoay pha cho BF2II.

```
1 assign phase_sel_i  = current_cnt[LOG2_DII];
2 assign phase_sel_ii = current_cnt[LOG2_DII];
3 assign rot_sel      = {1'b0, ~phase_sel_i};
```

Listing 6.10: Sinh tín hiệu chọn pha và chọn xoay pha

Bộ đếm cục bộ là thay đổi ở mức hiện thực RTL. Nó không làm thay đổi thuật toán Radix-2² đã trình bày trước đó, mà chỉ giúp lịch điều khiển bền hơn khi luồng `valid` có thể bị ngắt.

6.4.4 Sinh địa chỉ hệ số xoay

Địa chỉ đọc hệ số xoay được suy ra từ chỉ số nội bộ của khối. Sau khi bù độ trễ đi qua BF2I và BF2II, mạch tách chỉ số thành n_3 và q . Với $q = 0$, hệ số bằng 1 nên địa chỉ có thể đặt về 0. Với các giá trị còn lại, địa chỉ tương ứng lần lượt với n_3 , $2n_3$ hoặc $3n_3$.

```
1 logic [LOG2_DII-1:0] n3;
2 logic [1:0] q;
3
```

```

4 assign n3 = current_idx[LOG2_DII-1:0];
5 assign q  = current_idx[LOG2_DII+1:LOG2_DII];
6
7 always_comb begin
8     case (q)
9         2'd0: tw_addr_comb = '0;
10        2'd1: tw_addr_comb = TW_ADDR_W'(n3 << 1);
11        2'd2: tw_addr_comb = TW_ADDR_W'(n3);
12        2'd3: tw_addr_comb = TW_ADDR_W'((n3 << 1) + n3);
13    endcase
14 end

```

Listing 6.11: Giải mã địa chỉ đọc hệ số xoay

Phần này là nơi rất dễ sai nếu chỉ nhìn công thức tổng quát mà không xét thứ tự dữ liệu đi ra từ SDF. Vì vậy, việc có bản FFT-16 thu nhỏ để kiểm tra thứ tự hệ số xoay là cần thiết trước khi ghép lên FFT-256.

6.5 Từ FFT-16 thử nghiệm đến FFT-256 hoàn chỉnh

6.5.1 Bộ FFT-16 dùng để kiểm tra nhanh

Bộ FFT-16 được dùng như một phiên bản thu nhỏ để kiểm tra quy luật điều khiển trước khi chạy thiết kế 256 điểm. Với $N = 16$, hệ thống gồm hai khối Radix-2²SDF. Khối đầu có độ trễ 8 và 4, có bộ nhân phức với hệ số W_{16} . Khối thứ hai có độ trễ 2 và 1, không cần bộ nhân phức.

```

1  r22sdf_block_controlled #(
2      .DATA_W(DATA_W),
3      .DELAY_I(8),
4      .DELAY_II(4),
5      .HAS_CMUL(1),
6      .TW_ADDR_W(4),
7      .TW_FILE("rom/tw16.hex")
8  ) blk0 (...);
9
10 r22sdf_block_controlled #(
11     .DATA_W(DATA_W),
12     .DELAY_I(2),
13     .DELAY_II(1),
14     .HAS_CMUL(0)
15 ) blk1 (...);

```

Listing 6.12: Ghép hai khối Radix-2²SDF trong FFT-16

Bản FFT-16 giúp phát hiện sớm các lỗi về thứ tự dữ liệu, độ trễ của `valid/sync`, cách sinh `rot_sel` và cách đọc hệ số xoay. Đây là bước trung gian hợp lý, vì nếu nhảy thẳng lên FFT-256, lỗi điều khiển sẽ khó khoanh vùng hơn nhiều.

6.5.2 Bộ FFT-256

Bộ FFT-256 là thiết kế chính của đề án. Hệ thống gồm bốn khối Radix-2²SDF mắc nối tiếp. Các cặp độ trễ lần lượt là (128,64), (32,16), (8,4) và (2,1). Ba khối đầu có bộ nhân phức, còn khối cuối bỏ qua bộ nhân phức.

```

1  logic b0_valid, b0_sync, b1_valid, b1_sync, b2_valid, b2_sync;
2  logic signed [DATA_W-1:0] b0_re, b0_im, b1_re, b1_im, b2_re, b2_im;
3
4  r22sdf_block_controlled #(
5      .DATA_W(DATA_W), .DELAY_I(128), .DELAY_II(64),
6      .HAS_CMUL(1), .TW_ADDR_W(8), .TW_FILE("rom/tw256.hex")
7  ) blk0 (
8      .clk(clk), .rst_n(rst_n),
9      .valid_in(valid_in), .sync_in(sync_in),
10     .din_re(din_re), .din_im(din_im),
11     .valid_out(b0_valid), .sync_out(b0_sync),
12     .dout_re(b0_re), .dout_im(b0_im)
13 );
14
15 r22sdf_block_controlled #(
16     .DATA_W(DATA_W), .DELAY_I(32), .DELAY_II(16),
17     .HAS_CMUL(1), .TW_ADDR_W(6), .TW_FILE("rom/tw64.hex")
18 ) blk1 (
19     .clk(clk), .rst_n(rst_n),
20     .valid_in(b0_valid), .sync_in(b0_sync),
21     .din_re(b0_re), .din_im(b0_im),
22     .valid_out(b1_valid), .sync_out(b1_sync),
23     .dout_re(b1_re), .dout_im(b1_im)
24 );
25
26 r22sdf_block_controlled #(
27     .DATA_W(DATA_W), .DELAY_I(8), .DELAY_II(4),
28     .HAS_CMUL(1), .TW_ADDR_W(4), .TW_FILE("rom/tw16.hex")
29 ) blk2 (
30     .clk(clk), .rst_n(rst_n),
31     .valid_in(b1_valid), .sync_in(b1_sync),
32     .din_re(b1_re), .din_im(b1_im),
33     .valid_out(b2_valid), .sync_out(b2_sync),
34     .dout_re(b2_re), .dout_im(b2_im)
35 );
36

```

```

37 r22sdf_block_controlled #(
38     .DATA_W(DATA_W), .DELAY_I(2), .DELAY_II(1), .HAS_CMUL(0)
39 ) blk3 (
40     .clk(clk), .rst_n(rst_n),
41     .valid_in(b2_valid), .sync_in(b2_sync),
42     .din_re(b2_re), .din_im(b2_im),
43     .valid_out(valid_out), .sync_out(sync_out),
44     .dout_re(dout_re), .dout_im(dout_im)
45 );

```

Listing 6.13: Đường dữ liệu chính của `fft256_r22sdf_top.sv`

Độ trễ của bản RTL hiện tại được tính bằng tổng độ sâu các tầng SDF cộng với độ trễ một chu kỳ của mỗi bộ nhân phức có chốt ngõ ra.

Bảng 6.1: Độ trễ phân bổ của các khối trong FFT-256

Khối	DELAY_I	DELAY_II	Bộ nhớ hệ số	Có CMUL	Độ trễ
0	128	64	W_{256}	Có	$128+64+1 = 193$
1	32	16	W_{64}	Có	$32 + 16 + 1 = 49$
2	8	4	W_{16}	Có	$8 + 4 + 1 = 13$
3	2	1	—	Không	$2 + 1 = 3$
Tổng					258 chu kỳ

Con số 258 chu kỳ gồm $N - 1 = 255$ chu kỳ từ các đường trễ SDF và 3 chu kỳ từ ba bộ nhân phức có chốt ngõ ra. Nếu sau này thêm thanh ghi đường ống để cải thiện định thời, độ trễ có thể tăng, nhưng thông lượng vẫn có thể giữ ở mức một mẫu phức mỗi chu kỳ.

6.6 Kiểm chứng chức năng RTL

Quy trình kiểm chứng dùng testbench để đưa dữ liệu vào, chờ `valid_out/sync_out`, sau đó so sánh dữ liệu đầu ra với các tập tin kết quả đối chiếu. Mô hình đối chiếu được viết bằng Python/NumPy, nhưng không dùng kết quả số thực lý tưởng một cách thô. Thay vào đó, mô hình có lượng tử hóa Q1.15, có chia tỷ lệ giống RTL và có sắp xếp lại thứ tự đầu ra để phù hợp với dạng đảo thứ tự của DIF.

```

1 if (valid_out) begin
2     if (sync_out && latency == -1) begin

```

```

3      latency = cycle - start_cycle;
4      if (latency != EXP_LATENCY) begin
5          $display("ERROR: latency mismatch");
6          errors++;
7      end
8  end
9
10     exp_re = exp_out_dr[out_count][31:16];
11     exp_im = exp_out_dr[out_count][15:0];
12
13     if ((abs(dout_re - exp_re) > TOL) ||
14         (abs(dout_im - exp_im) > TOL)) begin
15         $display("ERROR: output mismatch at %0d", out_count);
16         errors++;
17     end
18     out_count++;
19 end

```

Listing 6.14: Khung kiểm tra độ trễ và dữ liệu trong testbench

Bảng 6.2: Tóm tắt các bài kiểm tra chức năng của FFT-256 RTL

Bài kiểm tra	Mục đích	Kết quả	Ghi chú
Mảng toàn số 0	Kiểm tra trạng thái tĩnh và đặt lại đường ống.	Đạt	Sai số 0 LSB.
Xung đơn tại mẫu 0	Kiểm tra phổ có biên độ đều trên các bin.	Đạt	Biên độ đồng đều sau khi chia tỷ lệ.
Tín hiệu một chiều không đổi	Kiểm tra năng lượng tập trung tại bin 0.	Đạt	Sai số 0 LSB.
Sóng đơn tại bin 1, 7, 31	Kiểm tra ánh xạ bin và thứ tự đảo dữ liệu.	Đạt	Xác nhận thứ tự đảo bit/đảo chữ số.
Dữ liệu phức ngẫu nhiên	Kiểm tra trường hợp tổng quát, tránh chỉ đúng với vector đặc biệt.	Đạt	Sai số lớn nhất khoảng 5 LSB.
20 khung liên tiếp	Kiểm tra nhiều khung dữ liệu liên tục.	Đạt	Không thấy rò dữ liệu giữa các khung.
3 khung nối liền	Kiểm tra khả năng xử lý luồng liên tục.	Đạt	valid/sync giữ đồng bộ.

Theo kết quả kiểm chứng, sai số lớn nhất ghi nhận khoảng 5 LSB ở phần thực và 4 LSB ở phần ảo; sai số trung bình tuyệt đối xấp xỉ 0.70 LSB. Mức sai số này phù

hợp với thiết kế số cố định có lượng tử hóa hệ số xoay và cắt bit sau bộ nhân phức. Tuy nhiên, kết quả này chỉ chứng minh đúng chức năng trong phạm vi các vector đã kiểm tra. Nó không phải kiểm chứng hình thức và cũng không thay thế cho kết quả tổng hợp tài nguyên, định thời hoặc công suất.

6.7 Quan sát dạng sóng bằng GTKWave

6.8 Giới hạn hiện tại và hướng hoàn thiện

Kết quả mô phỏng cho thấy thiết kế hoạt động đúng chức năng với các nhóm dữ liệu đã kiểm tra. Tuy nhiên, để tránh thổi phồng kết luận, cần ghi rõ ba giới hạn sau:

1. Chưa có kết quả tổng hợp FPGA hoặc ASIC, nên chưa thể kết luận số LUT, số thanh ghi, số DSP, số khối nhớ, tần số tối đa hoặc công suất tiêu thụ thực tế.
2. Chưa có kiểm chứng hình thức, nên chưa thể khẳng định thiết kế đúng với mọi chuỗi đầu vào có thể xảy ra.
3. Đường trễ hiện tại ưu tiên sự rõ ràng để dễ quan sát và gỡ lỗi. Với bản tối ưu tài nguyên, các đường trễ dài nên được chuyển sang bộ nhớ vòng, bộ nhớ phân tán hoặc bộ nhớ khối tùy nền tảng FPGA.

Nói cách khác, bản RTL hiện tại đủ để chứng minh thuật toán, lịch điều khiển và đường dữ liệu chính, nhưng chưa phải bản tối ưu cuối cùng cho triển khai phần cứng thực tế.

6.9 Tổng kết chương

Chương này đã trình bày quá trình hiện thực kiến trúc FFT-256 R²SDF bằng SystemVerilog. Các khối cơ bản gồm `trivial_rotator`, `butterfly_radix2_scaled`, `complex_multiplier_q15`, `sdf_feedback_delay`, `twiddle_rom` và các tầng SDF đã được ghép thành bốn khối Radix-2²SDF hoàn chỉnh.

Về sự thống nhất với các chương trước, RTL giữ nguyên các tham số cốt lõi: $N = 256$, bốn cụm Radix-2², ba bộ nhân phức đầy đủ, tổng đường trễ $N - 1$ mẫu phức, dữ liệu Q1.15 và đầu ra ở dạng đảo thứ tự. Điểm khác biệt chính là bản RTL dùng bộ đếm cục bộ thay vì chỉ dùng một bộ đếm toàn cục. Đây là thay đổi ở mức hiện thực nhằm giữ đúng pha điều khiển khi `valid` bị ngắt, không làm thay đổi bản chất thuật toán.

Các bài kiểm chứng bằng testbench và mô hình đối chiếu cho thấy hệ thống đạt kết quả đúng trên nhiều nhóm dữ liệu, gồm mảng 0, xung đơn, tín hiệu một chiều, sóng

đơn, dữ liệu phức ngẫu nhiên và nhiều khung liên tiếp. Sai số LSB nằm trong mức hợp lý đối với thiết kế số cố định.

Tài liệu tham khảo

- [1] S. He and M. Torkelson, “A New Approach to Pipeline FFT Processor,” *Proceedings of IPPS 1996*, IEEE, 1996.
- [2] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [3] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2010.
- [4] L. R. Rabiner and B. Gold, *Theory and Application of Digital Signal Processing*, Prentice-Hall, 1975.
- [5] E. H. Wold and A. M. Despain, “Pipeline and Parallel-Pipeline FFT Processors for VLSI Implementation,” *IEEE Transactions on Computers*, vol. C-33, no. 5, pp. 414–426, 1984.
- [6] G. Bi and E. V. Jones, “A Pipelined FFT Processor for Word-Sequential Data,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 37, no. 12, pp. 1982–1985, 1989.